



TALENTLOCK

EXCLUSIVE TALENT INFRASTRUCTURE

TECHNICAL ARCHITECTURE & EXECUTIVE BRIEF

System Architecture & Component Reference

A dense architectural assessment of the TalentLock platform as it exists on the **main** branch — a **modular monolith** spanning a React 19 single-page client, an Express 5 API server, and a contract-first OpenAPI codegen pipeline, with **20 discrete AI inference surfaces** metered against per-plan token quotas.

48

SHIPPED
FEATURES

179

API ROUTE
HANDLERS

31

POSTGRES
TABLES

20

AI INFERENCE
SURFACES

~100k

LINES OF
TYPESCRIPT

REPOSITORY / BRANCH

jeffy555/talentlock · main @ 23d405b

DOCUMENT CLASS

Architecture Reference · Confidential

REVISION

18 August 2026

PART

Table of Contents

Four analytical parts and three appendices. Every claim in this document is traceable to source files on the `main` branch.

01

Executive Summary & Application Overview

Purpose, target audience, core value proposition, current system state snapshot, technology stack confirmation, and primary engineering metrics.

1.1 Platform Thesis

1.2 Audience Segments

1.3 Value Proposition

1.4 System State Snapshot

1.5 Technology Stack

1.6 Engineering Metrics

02

02

Application Flow & Interactive User Journeys

Step-by-step primary workflows for both roles, the canonical request lifecycle, and end-to-end data flow mapping from client interface through every system layer to persistence and external services.

2.1 Role Topology

2.2 Employer Journey

2.3 Freelancer Journey

2.4 Request Lifecycle

2.5 AI Inference Data Flow

2.6 Autonomous Agent Loops

2.7 Async & Scheduled Flows

06

03

System Architecture & Feature Breakdown

Structural pattern analysis, layered topology, monorepo composition, the contract-first codegen loop, persistence design, and the complete catalogue of 48 implemented feature domains.

3.1 Pattern Analysis

3.2 Layer Topology

3.3 Monorepo Composition

3.4 Contract-First Pipeline

3.5 Persistence Design

3.6 Feature Catalogue

3.7 Cross-Cutting Concerns

16

04

Component Deep Dive — Inputs & Outputs

Granular per-component analysis of inputs, internal processing logic, and outputs. Interface definitions, core endpoints, data schemas, state management mechanisms, external dependencies, and boundary configuration.

4.1 Boundary Layer

4.2 Identity & Access

4.3 Domain Services

4.4 AI Inference

4.5 Autonomous Evaluators

4.6 Notification Fabric

4.7 Storage & Documents

4.8 Metering

4.9 Client Composition

4.10 External Boundaries

25

A

Appendices

Complete API surface index, environment configuration matrix, and the verification & quality posture.

A. API Surface Index · 36

B. Environment Matrix · 43

C. Verification Posture · 45

36

READING CONVENTION

Monospaced tokens such as `artifacts/api-server/src/lib/` denote verbatim repository paths. HTTP method chips (`GET` `POST` `PATCH` `PUT` `DEL`) and access pills (`PUBLIC` `AUTH` `EMPLOYER` `FREELANCER` `ADMIN` `ENTERPRISE` `MACHINE`) carry consistent meaning throughout. Teal indicates data paths, API surfaces, and critical components.

PART
01

Executive Summary & Application Overview

What the platform does, who it serves, why the architecture is shaped the way it is, and the measurable state of the system on the `main` branch.

1.1 Platform Thesis

TalentLock is a **vertically integrated freelance engagement platform** that compresses the entire hiring lifecycle — discovery, verification, negotiation, contracting, execution, and retrospective — into a single transactional system of record.

Conventional talent marketplaces optimise for **listing volume**: they match supply to demand and withdraw once contact is established, leaving contracting, exclusivity, and compliance to email and PDF attachments. TalentLock inverts that model. Its central architectural premise is the **exclusivity lock** — a booked freelancer is atomically marked unavailable, availability blocks are auto-generated, and the engagement is bound to a machine-generated, dual-signed legal agreement before work begins. Every downstream feature is a consequence of that premise.

The second organising premise is that **AI is infrastructure, not a feature**. Twenty discrete inference surfaces are embedded directly in domain workflows — document verification, agreement drafting, contract health scoring, rate benchmarking, interview briefing, post-engagement debriefing, and two autonomous outreach agents. Each is metered through a single accounting seam (`token_usage`) so cost attribution is a first-class property of the schema rather than an afterthought.

ARCHITECTURAL CONSEQUENCE

Because exclusivity and contract state are authoritative in Postgres — not in a third-party document service — the system requires strict transactional discipline. Quota enforcement, verification-level recalculation, and completeness scoring all execute inside Drizzle transactions using `SELECT ... FOR UPDATE` row locks. Conversely, every non-authoritative side effect (notifications, email, AI generation, analytics) is dispatched **fire-and-forget** and can never delay or fail a user-facing write.

1.2 Audience Segments

Four distinct principals hold non-overlapping capability sets. Role is resolved server-side from the `users.role` column; plan entitlement from `subscriptions.plan`; and the administrative principal is authenticated on an entirely separate credential path.

PRINCIPAL	AUTH PATH	PRIMARY OBJECTIVE	CAPABILITY SURFACE
Employer EMPLOYER	Clerk	Source, vet, lock, and contract scarce specialist talent with exclusivity guarantees.	Talent Vault browse/filter, AI match chat, watchlist, job posting, bookings and rate negotiation, agreement generation or upload, e-signature, discovery meetings with AI briefs, hiring dispositions, spend & hiring analytics, autonomous TalentSearch agent.
Freelancer FREELANCER	Clerk	Establish verifiable credibility and convert inbound demand without bidding wars.	Profile & teaching-credential management, document verification, portfolio, availability calendar, express interest, proposal generation, counter-offer negotiation, agreement summarisation and signing, earnings intelligence, autonomous Cruise Mode agent.
Enterprise Team ENTERPRISE	Clerk + plan gate	Coordinate multi-seat hiring against a shared pipeline and consolidated reporting.	Team creation, seat invitation with single-use expiring tokens, shared shortlist (supersedes personal watchlist), team-level spend and hiring analytics, custom agreement clauses.
Platform Operator ADMIN	HMAC cookie	Adjudicate identity verification and maintain platform integrity.	Freelancer and employer document review queues with signed-URL preview, manual verify/reject verdicts, platform statistics, user and subscription inspection, audit-log review, token-usage attribution, destructive data operations.

ARCHITECTURAL CONSTRAINT — ISOLATED ADMIN TRUST DOMAIN

The administrative principal never traverses Clerk. It authenticates against `ADMIN_USERNAME` / `ADMIN_PASSWORD` and receives an HMAC-SHA256 signed `tl_admin` cookie with an 8-hour TTL, signed with `SESSION_SECRET`. All mutating admin routes are additionally protected by a `csrf-csrf` double-submit token fetched from `GET /api/admin/csrf-token`. Login is rate-limited to 10 attempts per 5 minutes per IP, and **fails closed**: if `ADMIN_PASSWORD` is unset, admin login is disabled entirely rather than defaulting open.

1.3 Core Value Proposition

EXCLUSIVITY AS A PRIMITIVE

A confirmed booking **atomically** flips freelancer availability, writes an `availability_blocks` row, and gates agreement generation until both parties converge on rate. Exclusivity is enforced in the schema, not by convention.

VERIFIABLE TRUST

Bilateral verification: freelancers submit identity and credential documents, employers submit business registration. Both produce a **tiered badge** recalculated atomically on every status change, with credential expiry tracked on a 90/30/7-day alert ladder.

CONTRACT LIFECYCLE OWNERSHIP

Agreements are AI-drafted or employer-uploaded, redlined, health-scored across five dimensions, plain-English summarised, dual-signed with stored signature images, and emitted as a formatted PDF — **without leaving the platform**.

AUTONOMOUS DEMAND MATCHING

Two symmetric background agents — Cruise Mode (freelancer-side) and TalentSearch (employer-side) — evaluate every new job post or profile update and dispatch AI-authored direct messages under **strict rate, budget, and duplicate ceilings**.

1.4 Current System State Snapshot

Runtime & Platform Posture				
BRANCH / COMMIT main @ 23d405b	DEPLOYMENT TARGET Replit autoscale	RUNTIME Node 24 · ESM	DATASTORE PostgreSQL 16 (Neon)	TYPECHECK Passing
IDENTITY PROVIDER Clerk (proxied)	OBJECT STORAGE Azure Blob / GCS / local	INFERENCE PROVIDER OpenAI gpt-4o / 4o-mini	TRANSACTIONAL EMAIL Resend (optional)	BILLING Simulated (Stripe-shaped)

ARCHITECTURAL CONSTRAINT — COMMERCIAL GATES CURRENTLY OPEN

The environment flag `PREMIUM_FEATURES_FREE` (and its client twin `VITE_PREMIUM_FEATURES_FREE`) is **enabled**. All Growth, Pro, and Enterprise entitlements — team accounts, custom clauses, monthly token ceilings, and booking or job-post limits — are bypassed for every plan. Enforcement logic is fully implemented and unit-tested in `subscriptionGating.ts` and `plans.ts`; it is gated behind this single flag. Both flags must be set to `false` and list prices restored before paid launch. Checkout is a **simulated** upgrade shaped for Stripe; no payment processor is wired.

1.5 Technology Stack Confirmation

CLIENT TIER

- React 19.1.0
- TypeScript ES2022
- Vite 7.3
- Wouter 3.3
- TanStack Query 5.90
- Tailwind CSS 4.1
- Radix UI
- shadcn primitives 56
- Recharts
- Framer Motion
- react-hook-form + Zod
- lucide-react

SERVICE TIER

- Express 5
- Node 24
- Drizzle ORM 0.45
- Clerk @clerk/express
- Pino redacted
- Helmet
- csrf-csrf v4
- Zod 3.25
- @react-pdf/renderer
- pdf-parse · mammoth

CONTRACT, DATA & INTEGRATION TIER

- OpenAPI 3.x · 9,316 LOC
- Orval 8.x
- PostgreSQL 16
- drizzle-kit push
- OpenAI gpt-4o / 4o-mini
- Azure Blob Storage
- Google Cloud Storage
- Resend

BUILD, VERIFICATION & OPERATIONS TIER

- pnpm workspaces 10.33
- Vitest 70 suites
- Playwright E2E
- Testing Library
- GitHub Actions CI + cron
- Replit autoscale

1.6 Primary Engineering Metrics

~100_k LINES TYPESCRIPT Application + libs + scripts	179 ROUTE HANDLERS Across 40 router modules	31 POSTGRES TABLES 28 schema modules	20 AI SURFACES Metered token labels	48 FEATURE DOMAINS 37 written specifications
---	--	---	--	---

DIMENSION	COUNT	COMPOSITION
API route modules	40	<code>artifacts/api-server/src/routes/</code> — 39 mounted via <code>routes/index.ts</code> ; <code>verifyDocuments.ts</code> present but unmounted
HTTP handlers by verb	179	77 <code>GET</code> · 67 <code>POST</code> · 24 <code>PATCH</code> · 6 <code>DEL</code> · 5 <code>PUT</code>
Server library modules	66	<code>src/lib/</code> — AI prompt builders, evaluators, storage adapters, access control, metering, domain analytics
Client pages	29	27 routed via Wouter + <code>not-found</code> fallback + Clerk-hosted auth views
Client components	150	94 feature components across 16 domain directories + 56 <code>ui/</code> primitives
Database schema modules	28	<code>lib/db/src/schema/</code> declaring 30 tables + <code>token_usage</code> in <code>lib/db/schema.ts</code>
OpenAPI contract	9,316	Lines of <code>lib/api-spec/openapi.yaml</code> — single source of truth for both generated client hooks and server-side Zod validators
Automated test suites	78	23 server unit · 47 server integration · 8 client unit/component, plus a Playwright smoke journey
Written specifications	37	<code>spec/*/</code> directories, each carrying <code>features</code> · <code>clarify</code> · <code>plan</code> · <code>task</code> · <code>UI</code> · <code>validation</code>
Workspace packages	11	3 applications · 7 shared libraries · 1 script package

TECHNICAL NOTE — SPECIFICATION-DRIVEN DELIVERY

The repository carries 37 formal specification directories, each decomposed into `features.md` , `clarify.md` , `plan.md` , `task.md` , `UI.md` , and `validation.md` . Feature work is executed in a fixed phase order — **database** → **backend** → **frontend** → **admin** → **validation** — with codegen and typecheck gates between phases. This is the dominant reason a codebase of this feature density sustains a single passing typecheck across 11 packages.

PART
02

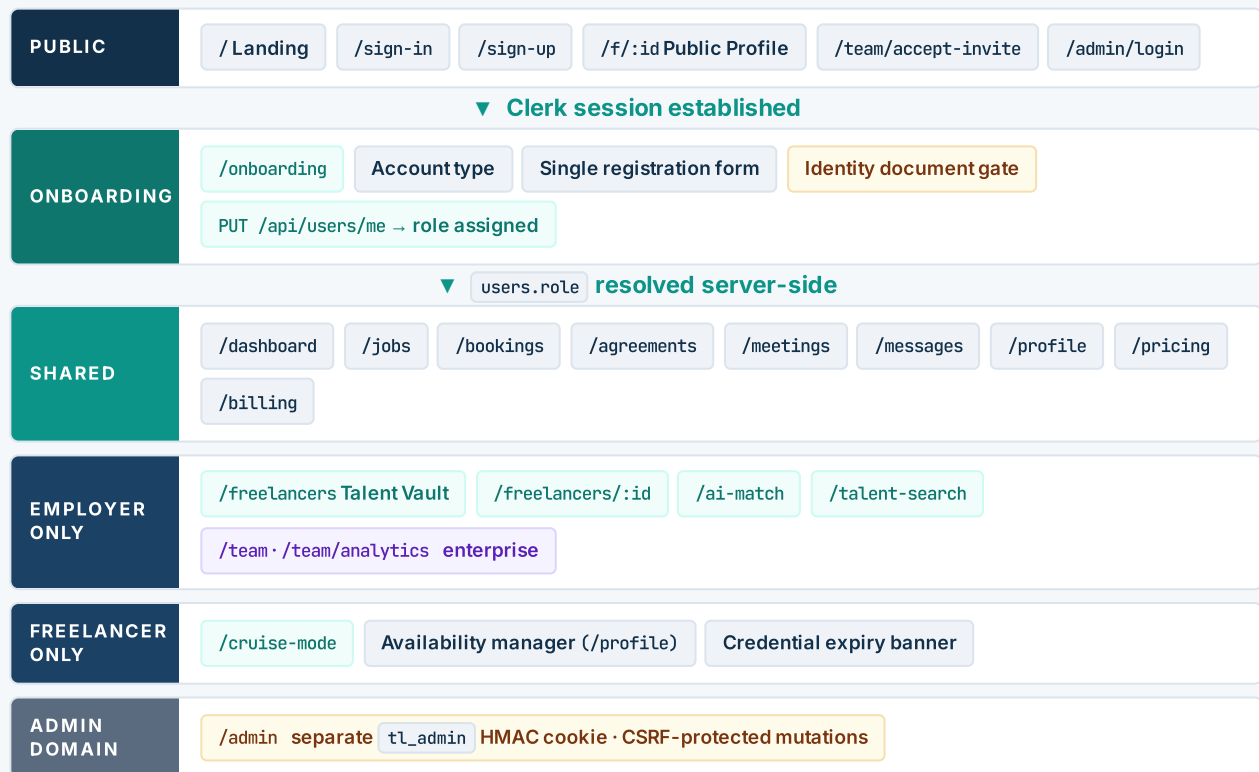
Application Flow & Interactive User Journeys

Primary workflows decomposed step by step, followed by end-to-end data flow mapping tracing a request from the client interface through every system layer to persistence and external services.

2.1 Role Topology & Entry Surfaces

The client is a single-page application whose route table is uniformly authenticated but **non-uniformly authorised**. Clerk's `ProtectedRoute` establishes identity; role and plan gates are applied at page level as redirect or inline denial, and the navigation shell hides inapplicable destinations based on `dbUser.role`.

FIGURE 2.1 — ENTRY SURFACES AND AUTHORISATION GATES



ARCHITECTURAL CONSTRAINT — REGISTRATION CANNOT COMPLETE WITHOUT IDENTITY

Both onboarding paths terminate in a **mandatory identity document upload** before `PUT /api/users/me` will assign a role. This forces a chicken-and-egg resolution: document routes must accept `users.role = 'pending'` when `onboardingRole` is set and a profile row exists. Onboarding therefore **must** call `PATCH /api/users/me/onboarding-step` first to materialise the pending user row.

2.2 Primary Workflow — Employer Engagement Lifecycle

The canonical revenue path: from talent discovery to a dual-signed, downloadable agreement and post-engagement retrospective.

1 Registration & business verification

Account type selection, then a single consolidated form capturing location (deriving currency), company profile, and a mandatory Representative ID upload. The document is queued for AI vision review; `verificationLevel` is recalculated atomically on every status change.

```
PATCH /users/me/onboarding-step → PUT /employers/me → POST /employer-documents/confirm → PUT /users/me
```

2 Talent discovery — three parallel modalities

Deterministic filtering in the Talent Vault (field, rate, availability window, keyword, profession category, teaching subject); conversational AI matching that reads job requirements and recommends candidates with explanations; or fully autonomous TalentSearch that evaluates profiles as they change. Vault visibility requires `completenessScore ≥ 60`.

```
GET /freelancers?q=&availableFrom= · POST /openai/conversations/:id/messages · GET /ai/match-explanation
```

3 Pipeline curation

Candidates are saved to a personal watchlist with private notes, plan-limited to 25 (Starter) or 100 (Growth). Active enterprise team members are redirected server-side to the shared `team_shortlist`, receiving `403` / `[]` on personal endpoints. Watchlist entries emit debounced alerts on availability or $\geq 5\%$ rate change.

```
POST /freelancers/:id/save · PATCH /freelancers/:id/watchlist · GET /freelancers/saved
```

4 Discovery meeting & AI brief

A meeting request transitions to `confirmed`, which **fire-and-forget** triggers a five-section AI brief — candidate snapshot, match rationale, suggested questions, rate context, watch points — cached as `briefContent` jsonb. The brief is employer-only and never exposed to the freelancer.

```
POST /meetings → PATCH /meetings/:id {status:confirmed} ~ generateMeetingBrief()
```

5 Hiring disposition

After completion the employer submits **internal** notes with a disposition. `next_round` produces an AI handoff summary routed to a team member or panel email; `proceed_to_booking` unlocks the Book CTA and upserts a candidate hiring file; `rejected` records the file only. One submission per meeting — `409` thereafter.

```
POST /meetings/:id/feedback {disposition} → employer_candidate_notes
```

6 Booking creation & rate convergence

Booking creation acquires a `FOR UPDATE` lock on the employer row to evaluate the active-booking ceiling before insert. Currency is **frozen** at creation alongside an `exchangeRateAtCreation` snapshot. Rate negotiation alternates proposals until `negotiationStatus` converges; agreement generation stays gated until then.

```
POST /bookings → POST /bookings/:id/negotiate · POST /ai/rate-suggestion
```

7 Agreement construction — generated or uploaded

Either GPT-4o drafts a contract from booking particulars and industry clause templates, or the employer uploads PDF/DOCX/TXT which is text-extracted and AI-summarised. The upload path advances

`summary_ready` → `enriched` → `finalized`; signing is blocked until `finalized`.

```
POST /agreements | POST /agreements/upload-url → upload-confirm → PATCH :id/amendments → POST :id/enrich → POST :id/finalize
```

8 Contract assurance

On-demand redlining proposes edits; a health score grades five dimensions (Clarity, Fairness, Completeness, Enforceability, Industry Fit) to a 0–100 composite and A–F letter. Accepting a redline **atomically invalidates both** the health score and freelancer summary caches in a single update.

```
POST /agreements/:id/redline → PATCH :id/accept-redline → POST :id/health-score
```

9 Execution & certified artefact

Both parties sign using stored signature images or typed cursive names. At `fully_signed` a formatted PDF is rendered via `@react-pdf/renderer` with both signatures, timestamps, and metadata, then cached to object storage on a fire-and-forget upload. Download is `403` unless fully signed.

```
POST /agreements/:id/sign → GET /agreements/:id/download
```


10 Delivery, closure & retrospective

Milestones track title, amount, due date, and status. On transition to **completed** a dual-perspective AI debrief is generated fire-and-forget; each party reads only their role-filtered slice. The employer then submits a star rating and comment, which the freelancer may answer once.

```
PATCH /bookings/:id {status:completed} ~ generateBookingDebrief() → GET /bookings/:id/debrief · POST /reviews
```

2.3 Primary Workflow — Freelancer Supply Lifecycle**1 Profile establishment & completeness gate**

Registration captures work category, location, professional details, and a mandatory identity document. An optional résumé import pre-fills fields but never satisfies the document gate. **calculateCompletenessScore()** is written in the **same update** as every profile change; below 60 the profile is invisible in the Talent Vault.

```
PATCH /freelancers/me · POST /freelancers/parse-resume · POST /documents/confirm
```

2 Credential verification & expiry stewardship

Documents receive GPT-4o vision adjudication returning **verified | rejected | needs_review** with a confidence score; low confidence escalates to the admin queue. Verified counts drive a three-tier badge. A daily scan advances an **expiryAlertStage** ladder (90d → 30d → 7d → expired) that only ever moves forward, making the scan idempotent and safe to miss a day.

```
POST /documents/confirm ~ triggerDocumentReview() · PATCH /documents/:type/expiry
```

3 Availability & portfolio curation

Manual blocks (holiday, unavailable) coexist with booking-derived auto-blocks, which are immutable — deletion returns **409**. **nextAvailableDate** is recomputed and surfaced on Vault cards and the **?availableFrom=** filter. Portfolio items and a public **/f/:id** profile support off-platform discovery.

```
POST /availability/me · DELETE /availability/me/:id · POST /portfolio
```

4 Demand capture — inbound and autonomous

Express interest in open postings (plan-limited, **FOR UPDATE** guarded), or delegate to Cruise Mode: rules are declared via form or free-form text parsed by AI, then every new job post is evaluated and an AI-authored DM sent on the freelancer's behalf above threshold — capped at 10 messages monthly with blackout windows and a dry-run mode.

```
POST /job-requirements/:id/interest · POST /cruise-mode/parse-rules → PATCH /cruise-mode/activate
```

5 Negotiation & contract comprehension

An AI proposal generator drafts responses in three tones. On the agreement, a **freelancer-only** summariser produces a six-section plain-English digest with up to three attention flags, disclaimer first. The endpoint returns **403** for employers and the panel is **not rendered at all** for them.

```
POST /ai/proposal · POST /agreements/:id/summarise · POST /agreements/:id/sign
```

6 Earnings intelligence

A six-month earnings trend against platform average, rate benchmarking percentile, forward monthly projection, and top-earning skills — all derived from completed bookings in the freelancer's own currency.

```
GET /dashboard/earnings-intelligence
```

2.4 Canonical Request Lifecycle

Every authenticated API call traverses a fixed, ordered middleware chain assembled in `artifacts/api-server/src/app.ts`. Order is load-bearing: body parsing must precede Clerk resolution, and the CSRF token endpoint must be registered before the CSRF gate it feeds.

01 helmet Security response headers	02 trust proxy Conditional on <code>TRUST_PROXY</code>	03 pino-http Query string stripped, fields redacted	04 clerk proxy <code>/api/__clerk</code> , production only	05 cors Allowlist, credentials enabled
06 cookieParser Required by admin cookie + CSRF	07 json / urlencoded Hard 1 MB body ceiling	08 clerkMiddleware Host-derived publishable key	09 csrf token route Registered before the gate	10 admin CSRF gate Non-GET under <code>/api/admin</code>

Handlers themselves are thin. The six-step interior sequence below is followed with near-total consistency across all 179 endpoints, and each step maps to a distinct failure code so client behaviour is predictable without reading server source.

#	HANDLER STEP	FAILURE	SEAM
1	Resolve Clerk identity	401	<code>getAuth(req)</code> — proves who , never what they own
2	Resolve internal user	404	<code>resolveUserByClerkId()</code> in <code>accessControl.ts</code>
3	Validate request shape	400	Generated Zod schema from <code>@workspace/api-zod</code>
4	Verify participation	403	<code>canAccessBooking / Meeting / Agreement / Conversation / JobRequirement</code>
5	Check entitlement	402	<code>checkLimit()</code> / <code>checkTokenQuota()</code> under a row lock
6	Sanitise and persist	500	<code>sanitiseText()</code> , then a Drizzle transaction

TECHNICAL NOTE — THE 402 CONTRACT

Commercial refusals are expressed as HTTP `402 Payment Required` with a uniform body `{ error, code, planNeeded }`. The client discriminates on `code`: `TOKEN_LIMIT` renders an **inline** error inside the originating drawer or panel, while `PLAN_LIMIT` **redirects** to `/pricing`. Mixing these produces dead-end drawers, so the distinction is enforced centrally in `artifacts/talentlock/src/lib/planErrorUtils.ts`.

FIGURE 2.4 — END-TO-END DATA FLOW, CLIENT INTERFACE TO PERSISTENCE AND BACK



Authoritative state is committed before the response; every derived or external effect is deferred. This is the single most consistent pattern in the codebase.

2.5 AI Inference Data Flow

All twenty inference surfaces share one pipeline shape. The invariant is that **token accounting is inseparable from inference**: usage is recorded on the same path as the model call, before the response is validated or persisted.

01 Quota probe <code>checkTokenQuota</code> under FOR UPDATE	02 Context assembly Domain rows joined and normalised	03 Prompt build Deterministic <code>*Utils.ts</code> builder	04 Model call gpt-4o or gpt-4o-mini	05 Meter <code>LogTokenUsage(label)</code>	06 Validate Shape guard, else safe fallback	07 Cache jsonb column on owning row
---	---	--	---	--	---	---

INFERENCE SURFACE	MODEL	TOKEN LABEL	CHARGED TO	RESULT PERSISTENCE
AI match chat	gpt-4o-mini	ai_match	Employer	<code>messages</code> rows
Match explanation	gpt-4o-mini	ai_match_explanation	Employer	Transient response
Agreement generation	gpt-4o	agreement_generation	Employer	<code>agreements.content</code>
Contract redlining	gpt-4o-mini	contract_redlining	Employer	Transient response
Contract health score	gpt-4o	contract_health_score	Employer	<code>healthScore</code> , <code>healthScoreDetail</code>
Agreement summary	gpt-4o	agreement_summary	Employer	<code>freelancerSummary</code>
Upload summary	gpt-4o	agreement_upload_summary	Employer	<code>employerSummary</code>
Upload enrich	gpt-4o	agreement_upload_enrich	Employer	<code>agreements.content</code>
Job description assistant	gpt-4o-mini	job_description_assistant	Employer	Transient response
Proposal generator	gpt-4o-mini	ai_proposal	Freelancer	Transient response
Rate suggestion	gpt-4o-mini	rate_suggestion	Employer	Transient response
Meeting brief	gpt-4o	meeting_brief	Employer	<code>meetings.briefContent</code>
Interview handoff summary	gpt-4o-mini	interview_handoff_summary	Employer	<code>meetings.feedbackSummary</code>
Booking debrief	gpt-4o-mini	booking_debrief	Employer	<code>bookings.debriefContent</code>
Freelancer doc review	gpt-4o vision	document_verification	System	<code>documents.status</code> , <code>confidence</code>
Employer doc review	gpt-4o vision	employer_doc_review	Employer ⁺	<code>employer_documents.aiNotes</code>
Cruise Mode rule parse	gpt-4o	cruise_mode_parse	Freelancer	<code>cruise_mode_configs.rules</code>
Cruise Mode evaluation	gpt-4o	cruise_mode_evaluation	Freelancer	<code>cruise_mode_activity</code>
TalentSearch rule parse	gpt-4o	talent_search_parse	Employer	<code>talent_search_configs.rules</code>
TalentSearch evaluation	gpt-4o	talent_search_evaluation	Employer	<code>talent_search_activity</code>

⁺ `employer_doc_review` is attributed to the employer for cost visibility but deliberately **excluded** from plan quota deduction — verification is a platform obligation, not discretionary consumption. `document_verification` is charged to an internal system user for the same reason.

Unmetered surface: `routes/resumeParser.ts` invokes OpenAI without a corresponding `LogTokenUsage` call — the sole gap in inference accounting.

OBSERVATION — METERING GAP

Résumé parsing (`POST /api/freelancers/parse-resume`) performs a `gpt-4o` call but does not record usage in `token_usage` . Every other inference path meters consistently. Consequences: résumé-import spend is invisible to cost reporting and unbounded by quota. Remediation is a single `LogTokenUsage` call with a new label, mirroring the established pattern.

2.6 Autonomous Agent Loops

Cruise Mode and TalentSearch are structural mirrors — one freelancer-side, one employer-side — sharing a two-stage filter, a governor stack, and DM delivery. Both are triggered by ordinary domain writes and are invisible to the triggering request.

PROPERTY	CRUISE MODE FREELANCER	TALENTSEARCH EMPLOYER
Trigger	<code>POST /job-requirements</code> — every new posting	<code>PATCH /freelancers/me</code> , <code>POST /freelancers</code> , and document verification when <code>completenessScore ≥ 60</code>
Backfill	—	Activation or dry-run scans existing Vault-visible profiles
Stage 1 — pre-filter	Deterministic: skills, rate band, exclusion keywords, blackout windows	Deterministic: profession category, education sub-type, rate band, required skills, excluded keywords, DBS, verified credentials
Stage 2 — evaluation	<code>gpt-4o</code> scores fit 0–100 and drafts the outreach message	<code>gpt-4o</code> scores fit 0–100 and drafts the outreach message
Budget governor	10 messages per month (<code>messagesThisMonth</code>)	6 evaluation-hours per day (<code>hoursUsedToday</code> , midnight-UTC reset)
Recipient governor	—	Max 3 notifications per freelancer per day across all employers
Duplicate suppression	Per job requirement	30-day window per (<code>employerId</code> , <code>freelancerId</code>) on <code>sent</code>
Delivery	AI message posted to a <code>human_direct</code> conversation via <code>sendAutomatedOutreachMessage()</code> , badged ✦, with in-app notification and email	
Observability	Every decision writes an activity row — including negative outcomes <code>prefilter_rejected</code> , <code>duplicate_skipped</code> , <code>ai_parse_failed</code> , <code>blackout</code> , <code>daily_limit_reached</code> , <code>dm_failed</code>	
Rehearsal	Dry-run mode evaluates and logs <code>dry_run_would_send</code> without dispatching any message	

FIGURE 2.6 — TALENTSEARCH EVALUATION LOOP WITH GOVERNOR STACK



Each refusal path is individually observable. Silent early-exits were removed so the employer-facing activity feed can never appear empty without explanation.

2.7 Asynchronous & Scheduled Flows

FLOW	TRIGGER	DISPATCH	EFFECT & GUARD
Meeting brief	status → confirmed	Fire-and-forget	Captures <code>previousStatus</code> before update; re-reads the meeting and exits if no longer confirmed, guarding the confirm-then-cancel race

FLOW	TRIGGER	DISPATCH	EFFECT & GUARD
Booking debrief	<code>status → completed</code>	Fire-and-forget	Re-reads booking and exits unless still completed; manual regeneration debounced 24 h via <code>debriefRegeneratedAt</code>
Document review	<code>POST /documents/confirm</code>	Fire-and-forget	Vision adjudication, then verification-level recalculation; on verify, re-triggers TalentSearch evaluation for the profile
Cruise Mode eval	<code>POST /job-requirements</code>	Fire-and-forget	Failure can never affect job creation
TalentSearch eval	Profile write / doc verify / activation	Fire-and-forget	Batch ceiling of 50 candidate configs per invocation
Watchlist alerts	<code>PATCH /freelancers/me</code>	Fire-and-forget	Fires on availability <code>false → true</code> or ≥5% rate change; debounced to one alert per pair per 24 h via <code>lastAlertAt</code>
Notifications	15 domain events	Fire-and-forget	<code>notifications</code> insert; client polls unread count every 30 s
Email	Paired with notifications	Fire-and-forget	Resend; suppressed if the recipient read the thread within 5 minutes; silent no-op when <code>RESEND_API_KEY</code> is unset
Audit logging	Sensitive actions	Fire-and-forget	<code>audit_logs</code> with IP, user agent, entity, metadata
Agreement PDF cache	First download	Fire-and-forget	Object-storage upload never blocks the byte stream to the client
Credential expiry scan	<code>MACHINE</code> <code>POST /api/cron/credential-expiry</code>	GitHub Actions, daily 07:00 UTC	Advances alert stages monotonically; flips lapsed documents to <code>expired</code> ; idempotent and safe to re-run or skip a day

ARCHITECTURAL CONSTRAINT — NO IN-PROCESS SCHEDULER

The deployment target is Replit **autoscale**, which idles and recycles instances; a resident `setInterval` timer would fire unpredictably or not at all. Scheduled work is therefore driven **externally** by a GitHub Actions workflow that POSTs to `/api/cron/credential-expiry` with a timing-safe `x-cron-secret` header. The route is deliberately mounted **outside** `/api/admin`, because the CSRF double-submit gate applied to that prefix is meaningless for machine-to-machine calls and would reject them. It fails closed: unset `CRON_SECRET` yields `500`, never an unauthenticated success.

PART
03

System Architecture & Feature Breakdown

Structural pattern analysis, layer topology, monorepo composition, the contract-first codegen loop, persistence design, and the complete catalogue of implemented capability.

3.1 Structural Pattern Analysis

TalentLock is a **Modular Monolith** deployed as two processes — a static single-page client and a single Express service — over one PostgreSQL database, organised inside a **pnpm workspace monorepo** with a contract-first API boundary.

CONFIRMED Modular Monolith with enforced internal seams

All 179 handlers execute in one process against one database. Modularity is achieved through directory and package boundaries rather than network boundaries: 40 route modules delegate to 66 single-responsibility library modules, and cross-cutting concerns are extracted into named seams (`accessControl.ts`, `subscriptionGating.ts`, `tokenLogger.ts`, `createNotification.ts`, `objectStorage.ts`). Route handlers orchestrate; they do not implement.

CONFIRMED Contract-first API boundary

`lib/api-spec/openapi.yaml` (9,316 lines) is the single source of truth. Orval generates React Query hooks into `lib/api-client-react/` and Zod validators into `lib/api-zod/` from that one artefact. The client cannot call an endpoint the contract does not describe, and the server validates against schemas derived from the same document — client/server drift is **structurally impossible**, not merely discouraged.

CONFIRMED Layered request pipeline

A uniform vertical sequence — boundary middleware → route handler → authorisation → validation → entitlement → domain library → ORM → deferred egress. Every handler follows it, which is why the codebase reads consistently across 48 feature domains.

CONFIRMED Deferred side-effect (fire-and-forget) model

Authoritative state commits synchronously; all derived effects — notifications, email, AI generation, audit logs, analytics, PDF caching — dispatch with `.catch()` and are never awaited. This is the system's substitute for a message broker: it delivers responsiveness and failure isolation without introducing queue infrastructure.

REJECTED Microservices

No service mesh, inter-service RPC, or per-service datastore. Given a single team, a shared relational core with heavy cross-entity joins (bookings ↔ agreements ↔ meetings ↔ profiles), and strict transactional invariants spanning those entities, distribution would impose distributed-transaction cost for no organisational benefit.

REJECTED Serverless / function-per-endpoint

The application is a long-lived Express process. Notably, the deployment target's autoscale idling **forced** scheduled work out of process — the credential-expiry scan is driven by an external GitHub Actions cron rather than an in-process timer.

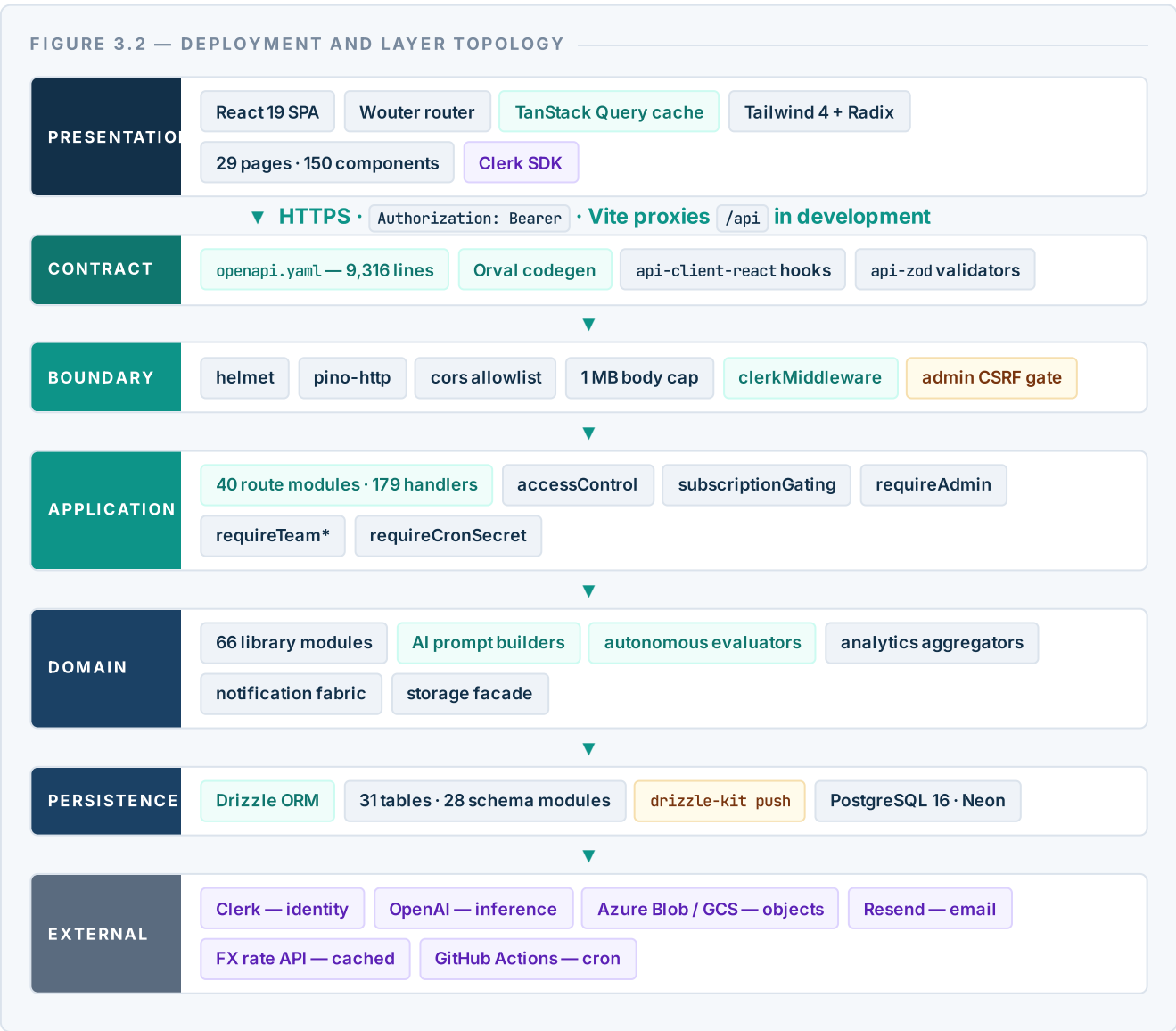
PARTIAL Clean / Hexagonal Architecture

Ports-and-adapters discipline is applied selectively where substitution has real value: `objectStorage.ts` abstracts Azure Blob, GCS, and local disk behind one interface. Elsewhere the ORM and framework types reach into domain modules, so this is a pragmatic partial adoption rather than a strict dependency-inversion architecture.

TECHNICAL NOTE — WHY THIS PATTERN FITS

The dominant architectural forces here are **transactional integrity** (exclusivity locks, quota ceilings, verification-level recalculation) and **contract stability** across a large, fast-growing API surface. A modular monolith with a generated contract addresses both directly. The two genuine scaling seams — AI inference and document processing — are already decoupled by the fire-and-forget model and could be extracted to workers without disturbing the request path.

3.2 Layer Topology



3.3 Monorepo Composition

PACKAGE	KIND	RESPONSIBILITY & BOUNDARY
@workspace/talentlock	App	React 19 SPA on Vite, dev port 25807. Proxies /api to localhost:8080, eliminating CORS configuration in development.
@workspace/api-server	App	Express 5 service on PORT (8080). Owns all business logic, authorisation, inference orchestration, and persistence access.
@workspace/mockup-sandbox	App	Isolated design-prototyping surface. Not part of the production runtime.
@workspace/db	Lib	Drizzle schema (28 modules), typed table exports, and the connection singleton. Provides push and a verify-schema parity guard.
@workspace/api-spec	Lib	Holds openapi.yaml and the Orval configuration. Its codegen script regenerates both downstream packages and then runs a library typecheck.
@workspace/api-client-react	Lib	Generated. React Query hooks plus custom-fetch.ts, which injects the Clerk bearer token via a registered token getter. Never hand-edited.

PACKAGE	KIND	RESPONSIBILITY & BOUNDARY
@workspace/api-zod	Lib	Generated. Zod request/response schemas consumed by server handlers. Configured with <code>indexFiles: false</code> ; index exports only <code>./generated/api</code> .
@workspace/integrations-openai-ai-server	Lib	Server-side OpenAI client wrappers — batch, image, and audio helpers.
@workspace/integrations-openai-ai-react	Lib	Client-side audio hooks (<code>useVoiceRecorder</code> , <code>useAudioPlayback</code>).
@workspace/object-storage-web	Lib	Uppy-based browser upload UI targeting presigned URLs.
@workspace/scripts	Ops	Operational tooling — demo seeding, destructive platform reset, document generation.

TECHNICAL NOTE — DEPENDENCY VERSION CATALOGUE

`pnpm-workspace.yaml` declares a **catalog** pinning shared dependency versions (React 19.1.0, Drizzle 0.45.2, Vite 7.3.2, TanStack Query 5.90.21, Zod 3.25.76, Tailwind 4.1.14, Wouter 3.3.5). Packages reference `catalog:` rather than literal ranges, so a version moves in exactly one place and cannot skew between the client and server. Platform-specific optional binaries (esbuild, rollup, Tailwind) are stripped via `overrides` to keep the lockfile portable.

3.4 Contract-First Codegen Pipeline

01 Edit contract <code>lib/api-spec/openapi.yaml</code>	02 Run Orval <code>api-spec run codegen</code>	03 Hooks emitted <code>api-client-react/generated</code>	04 Zod emitted <code>lib-zod/generated</code>	05 Implement route Handler + <code>routes/index.ts</code>	06 Typecheck gate 11 packages must pass
--	---	---	--	---	--

ARCHITECTURAL CONSTRAINT — MANDATORY POST-CODEGEN VERIFICATION

Orval regeneration is destructive and can silently reintroduce configuration drift. Three checks are mandatory after every run: `lib/api-zod/orval.config.ts` must still specify `indexFiles: false`; `lib/api-zod/src/index.ts` must export only `./generated/api`; and `pnpm run typecheck` must pass before any frontend work begins. Hand-editing anything under `lib/api-client-react/src/generated/` or `lib/api-zod/src/generated/` guarantees loss on the next codegen.

3.5 Persistence Design

Thirty-one tables across 28 schema modules. Schema evolution is **declarative**: `drizzle-kit push` diffs TypeScript definitions against the live database — there is no versioned SQL migration history in the repository.

DOMAIN	TABLES	DESIGN NOTES
Identity & profile	users · freelancer_profiles · employer_profiles	Profiles carry both <code>clerkId</code> and <code>userId</code> ; access-control helpers join on <code>userId</code> after resolving the user once. Location and currency are denormalised from <code>users</code> onto the freelancer profile and kept in sync by <code>syncFreelancerLocationFromUser()</code> .
Engagement core	job_requirements · job_interests · bookings · milestones	<code>bookings</code> carries negotiation state (<code>proposedRate</code> , <code>lastProposedBy</code> , <code>negotiationStatus</code>), a frozen <code>currencyCode</code> with <code>exchangeRateAtCreation</code> , and the <code>debriefContent</code> jsonb cache.
Contracting	agreements	Single wide table serving both <code>ai_generated</code> and <code>employer_upload</code> sources. Holds signature images and timestamps, plus four AI caches — health score, freelancer summary, employer summary, amendments — each with its own <code>*ScoredAt</code> invalidation timestamp.
Scheduling	meetings · availability_blocks · employer_candidate_notes	<code>meetings</code> carries the AI brief cache and internal hiring disposition fields that are never serialised to freelancers. Availability blocks are either manual or booking-derived; the latter are immutable.
Communication	conversations · messages · notifications	AI chat and human DMs share the same two tables, discriminated by <code>conversations.type</code> (<code>ai_match</code> <code>human_direct</code>) and <code>messages.senderType</code> . Added additively with NOT NULL defaults so all pre-existing AI rows backfilled without a data migration.
Verification	documents · employer_documents	Deliberately separate tables. Both unique on (<code>ownerId</code> , <code>documentType</code>) and upserted on re-upload, so exactly one row exists per type. Employer rows split <code>aiNotes</code> (admin-only) from <code>employerNotes</code> (employer-safe).
Autonomous agents	cruise_mode_configs · cruise_mode_activity · talent_search_configs · talent_search_activity	Structurally mirrored config/activity pairs. Configs hold jsonb <code>rules</code> plus a <code>rulesVersion</code> and budget counters; activity rows are an append-only decision ledger including negative outcomes.
Commerce & metering	subscriptions · token_usage · exchange_rate_cache	<code>token_usage</code> is the sole AI accounting seam — prompt, completion, and total tokens with a feature label and optional <code>conversationId</code> . FX rates are cached in-database; route handlers never call the external API directly.
Pipeline & teams	saved_freelancers · teams · team_members · team_shortlist	Personal watchlist and shared team shortlist are intentionally distinct. Active team members are blocked from personal endpoints server-side , not merely in the UI.

DOMAIN	TABLES	DESIGN NOTES
Trust & compliance	reviews · portfolio_items · audit_logs · account_deletion_requests	One review per booking with an optional single freelancer reply. Audit rows capture IP, user agent, entity reference, and metadata. GDPR deletion anonymises inside a transaction, then calls Clerk outside it, resetting to pending for retry on failure.

OPERATIONAL RISK — DECLARATIVE PUSH WITHOUT MIGRATION HISTORY

`drizzle-kit push` diffs local TypeScript against the live database. If a branch's schema files are **stale** relative to columns already pushed from another branch, the diff proposes **dropping live columns**. The repository mitigates this with `lib/db/verify-schema-parity.mjs` (`pnpm --filter @workspace/db run verify-schema`), which asserts a manifest of required column markers exists in both the local schema files and the target database before any push. The documented rule is absolute: reconcile with `main` first, and **never** use `push --force`.

3.6 Feature Catalogue

Forty-eight feature domains are implemented on `main`. Grouped below by capability cluster with the primary surfaces each occupies.

Discovery, Matching & Pipeline

#	FEATURE	ACCESS	IMPLEMENTATION SUMMARY
02	Talent Vault	EMPLOYER	Filter by field, rate, availability window, keyword, profession category, teaching subject. Visibility gated at <code>completenessScore ≥ 60</code> ; dual-currency display.
05	AI Talent Matching	EMPLOYER	Conversational GPT chat recommending candidates from job requirements; persisted as <code>ai_match</code> conversations.
21	Smarter Matching	EMPLOYER	Per-candidate explanation of why a recommendation was made across skills, rate, and availability.
11	Portfolio	FREELANCER	Portfolio items with images, URLs, and tags.
12	Public Profiles	PUBLIC	Unauthenticated <code>/f/:id</code> pages with read-only availability calendar.
13	Availability Calendar	AUTH	Manual and booking-derived blocks; <code>nextAvailableDate</code> badge; Vault <code>?availableFrom=</code> filter; auto-blocks immutable (<code>409</code> on delete).
43	Freelancer Watchlist	EMPLOYER	Personal pipeline with private notes; plan limits 25/100; debounced availability and ≥5% rate-change alerts; superseded by team shortlist for enterprise members.
37	TalentSearch	EMPLOYER	Autonomous employer-side agent — profile evaluation, AI DM dispatch, 30-day duplicate suppression, 6h/day budget, 3/day recipient cap, activation backfill.
35	Cruise Mode	FREELANCER	Autonomous freelancer-side agent — job evaluation, AI DM dispatch, 10 messages/month, blackout windows, dry-run mode.
48	Engagement Lists	AUTH	Shared list chrome across six surfaces — server-side <code>q</code> + status/decision filters, <code>pageSize: 10</code> , "Showing X-Y of Z", URL-synced state.

Engagement, Negotiation & Contracting

#	FEATURE	ACCESS	IMPLEMENTATION SUMMARY
03	Exclusive Bookings	AUTH	Booked freelancers receive a Lock badge and are marked unavailable atomically.
04	Rate Negotiation	AUTH	Alternating proposal/counter-proposal; agreement generation gated until convergence.
29	Smart Rate Suggestions	AUTH	Static market context on all plans plus AI recommendation on Growth+; "use suggested rate" fills the input only and never submits.
28	AI Proposal Generator	FREELANCER	Three-tone drafting drawer on pending bookings with a copyable accepted block.
09	Milestone Tracking	AUTH	Title, amount, due date, and status per booking milestone.
06	AI Agreement Generation	EMPLOYER	GPT-4o drafts contracts from booking particulars and industry clause templates.
46	Employer Uploaded Agreement	EMPLOYER	PDF/DOCX/TXT upload, text extraction, AI summary, amendment points, AI enrich, finalize gate before signing.
18	Agreement Templates	EMPLOYER	Industry-specific templates plus enterprise custom clauses.
19	Contract Redlining	EMPLOYER	AI review with edit suggestions; accepting one invalidates dependent AI caches atomically.
31	Contract Health Score	AUTH	0–100 composite across Clarity, Fairness, Completeness, Enforceability, Industry Fit with an A–F grade.
33	Agreement AI Summary	FREELANCER	Six-section plain-English digest with up to three attention flags, disclaimer first; 403 for employers and not rendered for them at all.
07	Signature Upload	AUTH	Stored handwritten signature images reused across all agreements.
08	Agreement Safe Locker	AUTH	Fully executed agreements unlock a certified per-party download.
34	Agreement PDF Download	AUTH	<code>@react-pdf/renderer</code> output with both signatures, timestamps, metadata; object-storage cached; requires <code>fully_signed</code> .
47	Job Timeline Validation	AUTH	End dates must be on or after start dates; enforced in both form and API.

Meetings, Trust & Verification

#	FEATURE	ACCESS	IMPLEMENTATION SUMMARY
38	AI Meeting Brief	EMPLOYER	Five-section brief on meeting confirmation, cached as jsonb; regeneratable via a 202 endpoint the client polls.
39	Discovery Meeting Outcome	EMPLOYER	Internal notes plus disposition; next-round AI handoff summary to team member or panel email; final dispositions upsert a candidate hiring file. Never candidate-facing.
16	Document Verification	FREELANCER	Vision AI adjudication with confidence; low confidence escalates to admin queue; tiered verified badge.
41	Employer Verification	EMPLOYER	Five business document types; verification level recalculated atomically; Representative ID mandatory at registration; Verified Employer badge.
42	Credential Expiry Tracking	MACHINE	Daily scan advancing a monotonic 90d → 30d → 7d → expired ladder; Vault exclusion narrowly scoped to expired school-teacher licences.

#	FEATURE	ACCESS	IMPLEMENTATION SUMMARY
10	Reviews & Ratings	AUTH	1–5 stars plus comment on completed bookings; averages on profiles; one freelancer reply.
36	Teaching Professional Profile	FREELANCER	Profession category plus 12 nullable education fields; conditional onboarding section; prompt injection byte-identical for technology roles.
44	Post-Engagement AI Debrief	AUTH	Dual role-specific debrief on completion; each party reads only their slice; manual regeneration on a 24-hour cooldown.

Platform, Commerce & Operations

#	FEATURE	ACCESS	IMPLEMENTATION SUMMARY
01	Dual Role System	AUTH	Account type then a single registration form; email and E.164 phone required; identity document gate before role assignment.
40	In-App Direct Messaging	AUTH	Human threads additively layered onto AI chat tables; 30 messages/hour per user per conversation; unread badge; read-state tracked server-side.
22	Notifications Centre	AUTH	15 event triggers, unread badge, 30-second polling, uniformly fire-and-forget.
45	Multi-Currency & Location	AUTH	Country-derived currency; booking currency frozen with FX snapshot; cached rate endpoint; currency-correct agreement rate clauses.
14	Subscription Tiers	AUTH	Five plans with enforced booking, job-post, and interest ceilings — currently open via PREMIUM_FEATURES_FREE .
17	AI Token Tracking	AUTH	Monthly quota per employer plan; per-conversation breakdown on Growth+.
30	Team Accounts	ENTERPRISE	Team creation, single-use 7-day invite tokens, shared shortlist, team analytics.
23	Earnings Intelligence	FREELANCER	Six-month trend vs platform average, rate percentile, projection, top earning skills.
24	Employer Spend Analytics	EMPLOYER	Spend trend, field breakdown, top freelancers, committed spend, rate vs market.
25	Hiring Analytics	EMPLOYER	Funnel conversion, skills demand/supply gap, retention, lifecycle duration, outcomes.
20	Job Description Assistant	EMPLOYER	AI generate / improve / check modes for job posts.
15	Admin Console	ADMIN	Stats, users, audit log, booking and job overview, document review queues, token attribution; CSRF-protected mutations.
26	Security Hardening	—	Helmet, 1 MB body caps, Pino redaction, universal text sanitisation, admin CSRF, audit logging, GDPR deletion.
32	Auth Hardening	—	Per-resource authorisation on 11 route families, storage ACL, server-derived signing role, namespaced upload paths, IDOR protection.
27	Product Gaps	—	Resend email with opt-out, Vault keyword search, completeness gate, paginated list endpoints, employer booking message, public profile preview.

3.7 Cross-Cutting Concerns

Security Posture

CONTROL	IMPLEMENTATION	BEHAVIOUR
Transport headers	helmet	Applied globally as the first middleware.
Origin control	cors allowlist	Explicit origins from <code>ALLOWED_ORIGINS</code> / <code>APP_URL</code> ; permissive only when unset in development.
Payload ceiling	express.json	Hard 1 MB limit on both JSON and urlencoded bodies.
Identity	@clerk/express	Global <code>clerkMiddleware</code> ; publishable key derived per host; Frontend API proxied at <code>/api/__clerk</code> in production.
Authorisation	accessControl.ts	Per-resource participation checks. Convention: unauthenticated <code>401</code> , authenticated non-participant <code>403</code> , unknown id <code>404</code> . Validated at 32/32 checks.
Admin identity	adminAuth.ts	HMAC-SHA256 <code>tl_admin</code> cookie, 8-hour TTL, <code>httpOnly</code> , <code>sameSite: strict</code> , <code>secure</code> in production.
CSRF	csrf-csrf v4	Double-submit <code>tl_csrf</code> cookie plus <code>x-csrf-token</code> header on all non-GET <code>/api/admin</code> requests.
Brute-force	routes/admin.ts	Admin login limited to 10 attempts per 5 minutes per IP, reset on success.
Machine auth	cronAuth.ts	Timing-safe <code>x-cron-secret</code> comparison; fails closed with <code>500</code> when the secret is unset.
Injection defence	sanitise.ts	<code>sanitiseText()</code> strips HTML on every free-text write; search terms pass through <code>sanitiseSearchQuery()</code> .
Object access	routes/storage.ts	Deny-by-default: only <code>documents/*</code> (admin session) and <code>uploads/{userId}/*</code> (owner only) are reachable; every other private prefix returns <code>403</code> .
Log hygiene	logger.ts	Pino redacts authorization headers, cookies, passwords, signatures, and Clerk ids; query strings stripped from logged URLs.
Auditability	auditLogger.ts	Sensitive actions recorded with IP, user agent, entity reference, and metadata.
Data subject rights	accountDeletion.ts	Active-booking guard, transactional anonymisation, then Clerk deletion outside the transaction with retry on failure.

OBSERVATION — NO GLOBAL RATE LIMIT

Rate limiting exists in exactly two places: admin login (10 per 5 minutes per IP) and human direct messages (30 per hour per user per conversation). There is **no** application-wide limiter on the Clerk-authenticated API surface, including the AI endpoints. Token quotas bound **cost** per account but not **request rate**, and those quotas are presently bypassed by `PREMIUM_FEATURES_FREE` . An authenticated client can therefore drive inference endpoints as fast as OpenAI will answer. A global limiter in front of `/api/ai/*` , `/api/openai/*` , and the agreement AI routes would close this before paid launch.

Entitlement & Metering

PLAN	ROLE	LIST PRICE	ENFORCED CEILINGS
freelancer_free	Freelancer	\$0	3 active bookings · 5 interests/month · no token quota
freelancer_pro	Freelancer	\$0 (was \$19)	10 active bookings · unlimited interests · Cruise Mode entitlement
employer_starter	Employer	\$49/mo	3 active bookings · 5 job posts/month · 50k tokens/month · 25 watchlist
employer_growth	Employer	\$0 (was \$199)	15 active bookings · 30 job posts/month · 250k tokens/month · 100 watchlist
employer_enterprise	Employer	Custom	Unlimited · team accounts · custom clauses · shared shortlist

Two enforcement mechanisms operate independently. `checkLimit()` counts domain rows against plan ceilings for job posts, active bookings, express interests, and team seats. `checkTokenQuota()` sums `token_usage.totalTokens` for the current month inside a transaction that first acquires `SELECT id FROM users WHERE id = $1 FOR UPDATE`, serialising concurrent inference requests so a burst cannot straddle the ceiling. Both are bypassed wholesale when `PREMIUM_FEATURES_FREE` is set, and the internal system user is always exempt.

Observability

- **Structured logging** — Pino via `req.log` in every handler; `console.log` is prohibited by convention. Sensitive fields are redacted at the logger, not at call sites.
- **Decision ledgers** — both autonomous agents append an activity row for every outcome, including refusals, making agent behaviour auditable without log archaeology.
- **Cost attribution** — `token_usage` supports per-user, per-feature, and per-conversation rollups, surfaced in both the user billing view and the admin console.
- **Audit trail** — `audit_logs` records authentication events and sensitive mutations with request provenance.
- **Gap** — no APM, distributed tracing, error aggregation, or uptime metrics are wired. Diagnosis currently depends on log inspection and database queries.

PART
04

Component Deep Dive — Inputs & Outputs

Granular component-by-component analysis. Each entry states its inputs, internal processing logic, outputs, and external dependencies or boundary configuration.

Components are grouped into ten functional strata. Every entry names its authoritative source path so any claim can be verified directly against the repository.

4.1 Boundary Layer

4.1.1 Application Assembly artifacts/api-server/src/app.ts · index.ts	
INPUTS	Raw HTTP requests; <code>PORT</code> , <code>TRUST_PROXY</code> , <code>ALLOWED_ORIGINS</code> , <code>APP_URL</code> , <code>CLERK_PUBLISHABLE_KEY</code> , <code>CLERK_SECRET_KEY</code> , <code>CSRF_SECRET</code> , <code>NODE_ENV</code> .
PROCESSING	Assembles the twelve-stage middleware chain in a fixed order, then mounts the aggregate router at <code>/api</code> . <code>index.ts</code> requires <code>PORT</code> and throws if absent — there is deliberately no in-code default. Registers <code>GET /api/admin/csrf-token</code> before applying <code>doubleCsrfProtection</code> to non- <code>GET /api/admin</code> traffic, since the gate depends on the token the route issues.
OUTPUTS	A configured Express 5 application; JSON responses; structured Pino request logs with query strings stripped and sensitive fields redacted.
BOUNDARY	<code>helmet</code> , <code>cors</code> , <code>cookie-parser</code> , <code>pino-http</code> , <code>@clerk/express</code> , <code>csrf-csrf</code> . Production-only Clerk Frontend API proxy mounted at <code>/api/_clerk</code> .

4.1.2 Router Aggregation artifacts/api-server/src/routes/index.ts	
INPUTS	Thirty-nine router modules imported explicitly.
PROCESSING	Mount order is semantically significant because Express matches greedily. Documented precedence rules: <code>GET /freelancers/saved</code> must precede <code>GET /freelancers/:id</code> ; <code>GET /availability/me</code> must precede <code>GET /availability/:freelancerId</code> ; <code>PATCH /notifications/read-all</code> must precede <code>PATCH /notifications/:id/read</code> . Violating any of these silently binds a literal segment as a route parameter.
OUTPUTS	A single composed <code>Router</code> exposing 179 handlers.
NOTE	<code>routes/verifyDocuments.ts</code> exists in the tree but is not imported — dead code that should be removed or mounted deliberately.

4.2 Identity & Access Control

4.2.1 Access Control src/lib/accessControl.ts	
INPUTS	Clerk id from <code>getAuth(req)</code> ; a target resource identifier.
PROCESSING	<code>resolveUserByClerkId()</code> maps the external identity to the internal <code>users</code> row exactly once per request. <code>canAccessBooking</code> , <code>canAccessMeeting</code> , <code>canAccessAgreement</code> , <code>canAccessConversation</code> , and <code>canAccessJobRequirement</code> then compare the caller's profile id against the resource's <code>employerId</code> / <code>freelancerId</code> . <code>agreementRoleForUser()</code> derives the signing role server-side so a client cannot assert which party it is signing as.
OUTPUTS	A resolved user record and a boolean participation verdict, mapped to the <code>401</code> / <code>403</code> / <code>404</code> convention.
COVERAGE	Eleven route families gated. Intentionally ungated public reads: freelancer list and detail, job list and detail, public profile, and public object serving.

4.2.2 Admin Authentication & CSRF src/lib/adminAuth.ts · src/lib/csrf.ts	
INPUTS	Credentials against <code>ADMIN_USERNAME</code> / <code>ADMIN_PASSWORD</code> ; the <code>tl_admin</code> cookie on subsequent requests; <code>x-csrf-token</code> header plus <code>tl_csrf</code> cookie on mutations.
PROCESSING	Issues <code>admin:{expiresAt}.{HMAC-SHA256-base64url}</code> signed with <code>SESSION_SECRET</code> . <code>requireAdmin</code> verifies signature and expiry; <code>isAdminRequest</code> provides a non-throwing variant for read paths such as private document serving. Login is rate-limited in-memory to 10 attempts per 5 minutes per IP.
OUTPUTS	An <code>httpOnly</code> , <code>sameSite: strict</code> cookie with an 8-hour TTL (<code>secure</code> in production); <code>401</code> JSON on failure.
CONSTRAINT	Fails closed — with <code>ADMIN_PASSWORD</code> unset, admin login is disabled rather than open. Entirely independent of Clerk; no Clerk session can escalate to admin.

4.2.3 Team & Machine Guards src/middleware/requireTeam.ts · src/lib/cronAuth.ts · src/lib/teamMembership.ts	
INPUTS	Clerk identity plus <code>subscriptions.plan</code> and <code>team_members</code> state; or an <code>x-cron-secret</code> header for machine callers.
PROCESSING	<code>requireEnterpriseEmployer</code> asserts the <code>employer_enterprise</code> plan; <code>requireTeamMember</code> and <code>requireTeamAdmin</code> assert an active membership row and role. <code>isActiveTeamMember()</code> is additionally consulted by the personal watchlist routes to divert enterprise members to the shared shortlist. <code>requireCronSecret()</code> performs a timing-safe comparison against <code>CRON_SECRET</code> .
OUTPUTS	Pass-through, or <code>402 PLAN_LIMIT</code> / <code>403</code> for team gates; <code>401</code> for a bad cron secret and <code>500</code> when it is unconfigured.
BOUNDARY	Cron routes are mounted outside <code>/api/admin</code> so the browser-oriented CSRF gate does not reject machine callers.

4.3 Domain Service Components

4.3.1 Bookings & Negotiation src/routes/bookings.ts · milestones.ts · lib/availabilityUtils.ts	
INPUTS	POST <code>/bookings</code> — freelancer id, job requirement, dates, rate, payment type, optional 500-char message. POST <code>/bookings/:id/negotiate</code> — proposed rate. PATCH <code>/bookings/:id</code> — status and milestone mutations.
PROCESSING	- Acquires <code>SELECT ... FOR UPDATE</code> on the employer row before evaluating the active-booking ceiling, preventing concurrent creates from straddling the limit. - Freezes <code>currencyCode</code> and stores <code>exchangeRateAtCreation</code> so historical spend analytics remain stable as FX moves. - Alternates <code>lastProposedBy</code> and <code>negotiationStatus</code> until convergence; agreement generation is gated on that state. - On confirmation, writes an <code>availability_blocks</code> row and refreshes <code>nextAvailableDate</code>; on cancellation, removes the derived block. - On <code>status → completed</code>, dispatches debrief generation fire-and-forget.
OUTPUTS	Paginated <code>{ data, total, page, pageSize, totalPages }</code> ; detail payloads exposing <code>hasDebrief</code> and <code>debriefGeneratedAt</code> but never raw <code>debriefContent</code> ; notifications to the counterparty.
DEPENDS ON	<code>accessControl</code> , <code>subscriptionGating</code> , <code>availabilityUtils</code> , <code>bookingDebriefGenerator</code> , <code>createNotification</code> , <code>countryData</code> .

4.3.2 Agreement Lifecycle src/routes/agreements.ts · lib/agreementPdf*.ts · lib/documentTextExtract.ts	
INPUTS	Booking id for AI generation; or an uploaded PDF/DOCX/TXT via presigned URL; amendment points; redline acceptances; signature image URL or typed name.
PROCESSING	Generated path — <code>gpt-4o</code> drafts from booking particulars plus industry clause templates, using the booking's frozen currency rather than a hardcoded one. Upload path — text extracted by <code>pdf-parse</code> (PDF), <code>mammoth</code> (DOCX/DOC), or UTF-8 (TXT), then AI-summarised for the employer. <code>uploadStage</code> advances <code>summary_ready → enriched → finalized</code>, and signing is refused until <code>finalized</code>. - Accepting a redline nullifies the health-score and freelancer-summary caches in a single update, so the two can never disagree. - Signing role is derived server-side; at <code>fully_signed</code> a PDF is rendered with both signatures and cached to object storage fire-and-forget. <code>enrichAgreementForViewer()</code> strips <code>employerSummary</code> for non-employer viewers.
OUTPUTS	Agreement rows with four independently-invalidated AI caches; a formatted PDF byte stream (<code>403 NOT_FULLY_SIGNED</code> otherwise); signature timestamps.
DEPENDS ON	<code>@react-pdf/renderer</code> , <code>pdf-parse</code> , <code>mammoth</code> , OpenAI, object storage, <code>contractHealthUtils</code> , <code>agreementSummaryUtils</code> , <code>agreementEnrichUtils</code> , <code>industryTemplates</code> .

4.3.3 Meetings & Hiring Disposition		src/routes/meetings.ts · lib/interviewHandoffSummary.ts
INPUTS	Meeting request fields; status transitions; POST <code>/meetings/:id/feedback</code> carrying internal notes and a disposition of <code>next_round</code> , <code>proceed_to_booking</code> , or <code>rejected</code> .	
PROCESSING	Captures <code>previousStatus</code> before the update so brief generation fires exactly once on the non-confirmed → confirmed edge. <code>next_round</code> produces an AI handoff summary routed to a team member or a panel email (mutually exclusive) and leaves the Book CTA closed. Final dispositions upsert <code>employer_candidate_notes</code> . Exactly one submission per meeting — subsequent attempts return <code>409</code> .	
OUTPUTS	Meeting rows with <code>briefContent</code> and disposition fields; candidate hiring file rows; team or panel handoff notification.	
HARD RULE	<code>feedbackText</code> , <code>feedbackSummary</code> , and panel fields are never returned to freelancers and never delivered as a candidate DM. Feedback is accepted only via the dedicated <code>/feedback</code> endpoint, never via PATCH .	

4.4 AI Inference Components

Every inference module separates **deterministic prompt construction** from the network call, which is what makes the prompt builders unit-testable without mocking OpenAI. Each pairs a `build*Prompt()` function with a `validate*Response()` guard.

4.4.1 Document Review (Vision) src/lib/documentReview.ts · visionImageUrl.ts · documentPdfPreview.ts	
INPUTS	Freelancer id, document type, and the stored object path for a confirmed upload.
PROCESSING	<code>resolveVisionImageUrl()</code> produces an image reference the model can actually fetch — in local development it reads the object and emits a base64 data URL , because OpenAI cannot resolve <code>localhost</code> signed URLs. PDFs are first rasterised to a first-page PNG. <code>gpt-4o</code> vision then returns <code>{ verdict, confidence, notes }</code> ; unparseable output degrades safely to <code>needs_review</code> rather than failing. <code>updateVerificationLevel()</code> recalculates the badge from verified counts, and a verified outcome re-triggers TalentSearch evaluation for that profile.
OUTPUTS	<code>documents.status</code> , <code>confidence</code> , <code>aiNotes</code> , <code>reviewedBy: 'ai'</code> ; freelancer <code>verificationLevel</code> and <code>isVerified</code> ; notification and email on a terminal verdict; usage recorded under <code>document_verification</code> against the system user.
DEPENDS ON	OpenAI vision, object storage, <code>systemUser</code> , <code>tokenLogger</code> , <code>talentSearchEvaluator</code> .

4.4.2 Meeting Brief Generator src/lib/meetingBriefGenerator.ts	
INPUTS	Meeting id. Internally joins the freelancer profile, the resolved job requirement, booking rate context, market medians, employer historical averages, and verified credentials.
PROCESSING	Re-reads the meeting and exits unless still <code>confirmed</code> , closing the confirm-then-immediately-cancel race. <code>resolveJobRequirement()</code> attempts three resolution paths; when none succeeds it emits a partial, data-only brief with generic questions and skips the model call entirely . Otherwise <code>gpt-4o</code> produces five sections. A deterministic budget comparison overrides the model's rate assessment, because arithmetic is authoritative and the model is not. The OpenAI client is lazily constructed so importing this module in tests does not require an API key.
OUTPUTS	<code>meetings.briefContent</code> jsonb and <code>briefGeneratedAt</code> ; employer notification and email; usage under <code>meeting_brief</code> charged to the employer.
BOUNDARY	Employer-only. Plan gating of the questions section is UI-only — the server always generates the full brief regardless of plan.

4.4.3 Booking Debrief Generator src/lib/bookingDebriefGenerator.ts	
INPUTS	Booking id; engagement history, milestones, review data, and rate context.
PROCESSING	Re-reads the booking and exits unless <code>completed</code> . <code>gpt-4o-mini</code> produces a dual debrief — separate employer and freelancer perspectives, including a rehire recommendation — stored as one jsonb document. Manual regeneration is debounced 24 hours via <code>debriefRegeneratedAt</code> ; the automatic first generation is not debounced.
OUTPUTS	<code>bookings.debriefContent</code> ; <code>202 Accepted</code> on manual regeneration; usage under <code>booking_debrief</code> charged to the employer.
CONSTRAINT	<code>GET /bookings/:id/debrief</code> returns a role-filtered slice only . Agreement <code>content</code> must never enter the debrief prompt. GDPR deletion nullifies the cache for participant bookings.

4.5 Autonomous Evaluator Components

4.5.1 TalentSearch Evaluator src/lib/talentSearchEvaluator.ts · talentSearchUtils.ts	
INPUTS	A freelancer profile id (from profile write, document verification, or activation backfill); all active <code>talent_search_configs</code> .
PROCESSING	<code>normaliseFreelancer()</code> flattens profile, education, rate, and verification state into one comparable shape. <code>talentSearchPreFilterReason()</code> returns a human-readable reason rather than a bare boolean, so rejections are explainable in the activity feed; <code>talentSearchPreFilter()</code> is a thin wrapper over it. - Governors applied in order — blackout window, 30-day duplicate, employer hour budget, recipient daily cap — each writing its own activity row on refusal. <code>gpt-4o</code> scores 0–100 with matched/concern/blocker reasons and drafts the message; any blocker forces a skip regardless of score. - Batch ceiling of 50 configs per invocation bounds worst-case fan-out.
OUTPUTS	A <code>talent_search_activity</code> row for every outcome — <code>sent</code> , <code>skipped</code> , <code>prefilter_rejected</code> , <code>duplicate_skipped</code> , <code>ai_parse_failed</code> , <code>blackout</code> , <code>daily_limit_reached</code> , <code>daily_freelancer_limit_reached</code> , <code>dm_failed</code> , <code>dry_run_would_send</code> — plus, on send, a <code>human_direct</code> message, recipient notification, employer notification, and an incremented daily counter.
DEPENDS ON	OpenAI, <code>automatedOutreachMessaging</code> , <code>cruiseModeUtils</code> (<code>isInBlackoutWindow</code> , <code>getNextMidnightUTC</code> reused directly), <code>tokenLogger</code> , <code>createNotification</code> .

4.5.2 Cruise Mode Evaluator src/lib/cruiseModeEvaluator.ts · cruiseModeUtils.ts	
INPUTS	A newly created job requirement id; all active <code>cruise_mode_configs</code> .
PROCESSING	Mirror of 4.5.1 with the polarity reversed. <code>normaliseJob()</code> then <code>preFilter()</code> on skills, rate band, and exclusions; blackout windows respected; monthly message budget of 10 enforced via <code>messagesThisMonth</code> with a scheduled reset. <code>gpt-4o</code> scores fit and drafts an interest message sent to the employer on the freelancer's behalf.
OUTPUTS	<code>cruise_mode_activity</code> rows; a badged <code>human_direct</code> message to the employer; usage under <code>cruise_mode_evaluation</code> charged to the freelancer.
CONSTRAINT	Dispatched fire-and-forget from <code>POST /job-requirements</code> ; evaluation failure must never affect job creation. Activation requires the <code>freelancer_pro</code> plan (<code>402 PLAN_LIMIT</code> otherwise).

4.5.3 Credential Expiry Scan src/lib/credentialExpiryScan.ts · credentialExpiryUtils.ts · routes/cron.ts	
INPUTS	<code>POST /api/cron/credential-expiry</code> with <code>x-cron-secret</code> ; all <code>documents.expiryDate</code> and <code>freelancer_profiles.teachingLicenceExpiry</code> values.
PROCESSING	<code>daysUntil()</code> and <code>targetStageForDaysRemaining()</code> compute the intended alert stage; <code>stageAdvanced()</code> permits forward-only transitions through <code>none → 90d → 30d → 7d → expired</code> . This monotonicity makes the scan idempotent — safe to run repeatedly and safe to miss a day and catch up. Lapsed verified documents flip to <code>expired</code> , and because <code>updateVerificationLevel()</code> counts only <code>verified</code> rows, the badge downgrades with no additional logic.
OUTPUTS	Advanced <code>expiryAlertStage</code> / <code>teachingLicenceAlertStage</code> values; status flips; staged notifications and emails; a summary result object.
BOUNDARY	Triggered by a GitHub Actions schedule (daily 07:00 UTC). Vault exclusion is narrowly scoped to expired school-teacher teaching licences and deliberately not generalised to all credential types.

4.6 Notification & Messaging Fabric

4.6.1 Notification & Email Dispatch src/lib/createNotification.ts · emailService.ts	
INPUTS	Target user id, notification type, entity type and id, message body. Profile-to-user helpers resolve recipients from employer or freelancer profile ids.
PROCESSING	Inserts a <code>notifications</code> row and optionally dispatches a Resend email. Both are invoked with <code>.catch()</code> and never awaited from a route handler. Email honours the per-user <code>emailNotificationsEnabled</code> opt-out and is suppressed when the recipient read the relevant thread within the last five minutes. With <code>RESEND_API_KEY</code> unset the sender is a silent no-op, so local development needs no mail configuration.
OUTPUTS	Notification rows consumed by <code>GET /notifications</code> and <code>/notifications/unread-count</code> , which the client polls every 30 seconds; outbound transactional email with deep links resolved through <code>APP_URL</code> .
COVERAGE	Fifteen distinct event triggers; <code>notificationRoutes.ts</code> on the client maps <code>entityType</code> to an in-app destination.

4.6.2 Conversations & Automated Outreach src/lib/conversationsUtils.ts · automatedOutreachMessaging.ts · routes/conversations.ts	
INPUTS	Employer and freelancer ids with optional booking or meeting scope; message content; or an agent-authored outreach payload from either evaluator.
PROCESSING	<code>findOrCreateConversation()</code> upserts against two unique partial indexes — one scoped to <code>bookingId</code> , one unscoped — so concurrent opens converge on a single thread. <code>sendHumanMessage()</code> writes the message, updates <code>lastMessageAt</code> , and dispatches notification and email. <code>shouldSuppressEmail()</code> applies the five-minute read window. Rate limiting counts messages from the sender in that conversation over the trailing hour and returns <code>429</code> at 30.
OUTPUTS	<code>conversations</code> and <code>messages</code> rows with <code>type='human_direct'</code> and <code>senderType='human'</code> ; unread counts; conversation and message ids returned to the evaluators for activity-row linkage.
CONSTRAINT	AI chat routes under <code>/api/openai/conversations/*</code> are a separate router and completely unchanged. Posting to <code>/api/conversations/:id/messages</code> for an <code>ai_match</code> conversation returns <code>422</code> . GDPR deletion anonymises message content to <code>[Message removed]</code> rather than deleting conversation rows.

4.7 Storage & Document Components

4.7.1 Object Storage Facade src/lib/objectStorage.ts · azureObjectStorage.ts · localObjectStorage.ts · objectAcl.ts	
INPUTS	Relative object keys; TTL requests for signed URLs; byte buffers; <code>AZURE_STORAGE_CONNECTION_STRING</code> , <code>PRIVATE_OBJECT_DIR</code> , <code>PUBLIC_OBJECT_SEARCH_PATHS</code> , <code>DEFAULT_OBJECT_STORAGE_BUCKET_ID</code> , <code>LOCAL_OBJECT_STORAGE_DIR</code> .
PROCESSING	A single interface — <code>getSignedUploadUrlForKey</code> , <code>getSignedReadUrlForKey</code> , <code>privateObjectExists</code> , <code>readPrivateObjectBuffer</code> — over three backends. Local development signs its own HMAC upload and read URLs served by dedicated routes, so the full upload flow works with no cloud credentials. This is the clearest ports-and-adapters boundary in the codebase.
OUTPUTS	Presigned upload and read URLs (documents default to a 15-minute read TTL); object byte streams; existence booleans used to validate uploads before database commit.
BOUNDARY	<code>routes/storage.ts</code> is deny-by-default : only <code>documents/*</code> (admin session) and <code>uploads/{userId}/*</code> (owner only) are reachable. Path namespacing keeps freelancer documents, employer documents, and agreements in separate prefixes.

4.7.2 Document Upload & Admin Review

src/routes/documents.ts · employerDocuments.ts · admin.ts ·
Lib/employerDocReviewUtils.ts

INPUTS	Document type, storage path, optional expiry date; admin verdicts with notes.
PROCESSING	Validates the storage path is namespaced to the calling user, then confirms the object actually exists via <code>privateObjectExists()</code> before any database write — preventing rows that point at absent files. Upsert on <code>(ownerId, documentType)</code> resets status, confidence, notes, and expiry state , so a renewed credential cannot inherit a stale <code>expired</code> stage. <code>recalculateEmployerVerificationLevel(tx, ...)</code> runs inside the same transaction as every status change, so level and documents never drift. Full verification requires all three of representative ID, company registration, and tax/VAT certificate.
OUTPUTS	Document rows; recalculated verification level and <code>isVerified</code> ; admin queue payloads including <code>signedFileUrl</code> ; notifications to the document owner.
CONSTRAINT	<code>aiNotes</code> is admin-only and never serialised to any owner-facing response; <code>employerNotes</code> is the employer-safe channel. <code>GET /employer-documents/me</code> must use an explicit field allowlist, never <code>SELECT *</code> . Admin rejection requires notes — <code>400</code> without them.

4.8 Metering & Entitlement Components

4.8.1 Token Logger & Subscription Gating

src/lib/tokenLogger.ts · subscriptionGating.ts · plans.ts · systemUser.ts

INPUTS	User id, feature label from the <code>TOKEN_FEATURES</code> enum, the OpenAI usage object, and an optional conversation id; plan id and current-period domain counts.
PROCESSING	<code>logTokenUsage()</code> appends prompt, completion, and total tokens to <code>token_usage</code> . <code>checkTokenQuota()</code> sums the current month inside a transaction that first takes <code>SELECT id FROM users WHERE id = \$1 FOR UPDATE</code> , so concurrent inference requests are serialised against the ceiling. <code>checkLimit()</code> counts domain rows against per-plan caps. Both short-circuit for the system user, unlimited plans, and whenever <code>PREMIUM_FEATURES_FREE</code> is set.
OUTPUTS	<code>token_usage</code> rows supporting per-user, per-feature, and per-conversation rollups; allow/deny verdicts carrying <code>planNeeded</code> ; <code>402</code> bodies shaped <code>{ error, code, planNeeded }</code> .
CONSUMERS	<code>GET /token-usage/me</code> , <code>GET /token-usage/conversation/:id</code> (Growth+), <code>GET /admin/token-usage</code> , and the billing view.

4.9 Client Composition & State Management

4.9.1 Application Shell artifacts/talentlock/src/App.tsx · main.tsx · components/layout/AppLayout.tsx	
INPUTS	<code>VITE_CLERK_PUBLISHABLE_KEY</code> , <code>VITE_CLERK_PROXY_URL</code> , <code>VITE_PREMIUM_FEATURES_FREE</code> , <code>BASE_PATH</code> ; the Clerk session; the current user record from <code>useGetMe</code> .
PROCESSING	Composes <code>ClerkProvider</code> → <code>QueryClientProvider</code> → <code>WouterRouter</code> (based at <code>import.meta.env.BASE_URL</code>). <code>ClerkAuthTokenSetter</code> registers a token getter with the generated fetch mutator so every hook attaches <code>Authorization: Bearer</code> without per-call wiring. <code>ProtectedRoute</code> enforces authentication; role and plan checks are page-level, and <code>AppLayout</code> hides inapplicable navigation by <code>dbUser.role</code> .
OUTPUTS	A rendered SPA across 27 routed pages with a shared navigation shell, notification bell, unread message badge, and toast surface.
NOTE	A new Clerk user intentionally receives <code>404</code> from <code>GET /users/me</code> ; the landing page and onboarding both treat that as “new user”. The query client is configured not to retry 4xx responses, which is what makes that signal usable.

4.9.2 Server State — Generated Query Layer lib/api-client-react/src/generated/ · custom-fetch.ts	
INPUTS	<code>openapi.yaml</code> at build time; hook parameters and request bodies at runtime.
PROCESSING	Orval emits one hook per operation plus a stable query-key factory (<code>getXQueryKey()</code>). All hooks funnel through <code>customFetch</code> , which resolves <code>baseUrl</code> to <code>/api</code> and attaches the bearer token when no explicit <code>Authorization</code> header is present. Mutations are followed by explicit <code>invalidateQueries()</code> calls against the generated keys — server state is never mirrored into component state .
OUTPUTS	Typed <code>data</code> , <code>isLoading</code> , <code>isPending</code> , and <code>error</code> values; a normalised cache that is the single source of truth for server-derived data.
CONSTRAINT	Never hand-edited. React Query hooks are never written by hand — a new endpoint requires a contract change plus codegen.

4.9.3 Client State Mechanisms artifacts/talentlock/src/hooks/ · src/lib/	
INPUTS	URL query strings, user interaction, viewport media queries, <code>sessionStorage</code> .
PROCESSING	Four state mechanisms, deliberately separated by lifetime: Server cache — TanStack Query, authoritative for all remote data. URL state — <code>useEngagementListQueryState</code> synchronises search, status, and page into the query string so list views are shareable and back-button correct; page resets to 1 whenever a filter changes. Ephemeral UI state — <code>useState</code> for drawers, dialogs, and form drafts; <code>use-toast</code> reducer for transient messages. Session-scoped state — <code>sessionStorage</code> for dismissed review prompts and AI-match job context.
OUTPUTS	Deterministic, shareable list views and predictable component behaviour across navigation.
CONSTRAINT	<code>localStorage</code> is prohibited in components — <code>sessionStorage</code> or React state only. AI output is never written into form state until the user explicitly accepts it, via an <code>onAccept</code> callback rather than the assistant mutating the parent form.

Component inventory by domain

DIRECTORY	COUNT	REPRESENTATIVE COMPONENTS
ui/	56	shadcn/Radix primitives — dialog, sheet, drawer, table, tabs, select, calendar, chart, sidebar, command, toast
(root)	33	AdminDocumentReviewTab, AdminEmployerDocumentTab, ContractHealthScoreCard, AgreementSummaryPanel, DocumentUploader, NotificationBell, RateSuggestionWidget, ProposalGeneratorDrawer, CredentialExpiryBanner, CompletenessBanner
onboarding/	7	CountryStateFields, PhoneWithCountryFields, FreelancerDocumentOnboardingStep, EmployerDocumentOnboardingStep, TeachingDetailsSection, ProfileStrengthChecklist
messages/	6	MessagesWorkspace, InlineMessageThread, BookingMessageThread, FloatingChatBox, ChatBoxProvider
earnings/ · spend/ · hiring/	19	Recharts-based analytics panels — trend charts, benchmark cards, funnels, retention, skills-gap, outcome donuts
availability/	5	AvailabilityCalendar (read-only), AvailabilityManager, AddBlockModal
agreements/	3	EmployerAgreementUploadPanel, EmployerAgreementSummaryPanel, EmployerAgreementWorkflow stepper
cruise-mode/ · talent-search/	6	Mirrored StatusBar, RuleBuilder, ActivityFeed triads
watchlist/ · employer/ · meetings/ · bookings/ · currency/ · profile/ · lists/	15	WatchListToggleButton, VerifiedEmployerBadge, MeetingBriefCard, MeetingOutcomeCard, DebriefCard, DualRateDisplay, EngagementListToolbar

4.10 External Dependencies & Boundary Configuration

INTEGRATION	BOUNDARY	CONFIGURATION	FAILURE MODE & DEGRADATION
Clerk	Identity	CLERK_SECRET_KEY, CLERK_PUBLISHABLE_KEY, VITE_CLERK_PROXY_URL	Hard dependency. Global middleware; Frontend API proxied at <code>/api/_clerk</code> in production. Also invoked during GDPR deletion — Clerk failure there resets the request to <code>pending</code> for retry rather than losing it.
OpenAI	Inference	OPENAI_API_KEY_TALENTLOCK	Degrades gracefully. Clients are lazily constructed; document review falls back to <code>needs_review</code> , evaluators log <code>ai_parse_failed</code> , and briefs fall back to data-only output. No AI failure breaks a domain write.
Azure Blob Google Cloud Storage	Objects	AZURE_STORAGE_CONNECTION_STRING, DEFAULT_OBJECT_STORAGE_BUCKET_ID, PRIVATE_OBJECT_DIR, LOCAL_OBJECT_STORAGE_DIR	Substitutable. Local disk backend with self-signed HMAC URLs means the entire upload path works without cloud credentials. Only file features are affected when storage is unavailable.
Resend	Email	RESEND_API_KEY, EMAIL_FROM, APP_URL	Optional. Silent no-op when unkeyed; in-app notifications remain the primary channel. Dispatch is fire-and-forget.

INTEGRATION	BOUNDARY	CONFIGURATION	FAILURE MODE & DEGRADATION
FX rate provider	Reference data	–	Cached and fallback-guarded. Rates are read from <code>exchange_rate_cache</code> ; handlers never call the upstream API directly, and hardcoded fallbacks cover a cold cache.
GitHub Actions	Scheduling	CRON_SECRET, API_BASE_URL	External scheduler. A missed run is self-correcting because alert stages advance monotonically and catch up on the next execution.
Stripe	Payments	stripeCustomerId, stripeSubscriptionId (columns only)	Not integrated. Columns exist and upgrade is simulated; the surface is shaped for Stripe but no processor is wired.



API Surface Index

All 179 handlers across 40 router modules, grouped by domain. Every path is prefixed `/api`.

Reference & health PUBLIC

VERB	PATH	PURPOSE
GET	/healthz	Liveness probe
GET	/countries	Country list with default currency and dial code
GET	/exchange-rates	Cached FX rates for dual-currency display
GET	/subscriptions/plans	Plan catalogue
POST	/demo/sign-in-token	Demo session mint — 404 unless non-production or explicitly enabled

Identity, users & account AUTH

VERB	PATH	PURPOSE
GET	/users/me	Current user; 404 signals a new Clerk user
PUT	/users/me	Complete registration; assigns role; requires email + E.164 phone
PATCH	/users/me/onboarding-step	Persist in-progress onboarding; upserts the pending user row
PATCH	/users/me/contact	Update contact email and phone
PATCH	/users/me/location	Update country/state and derive currency
PATCH	/users/me/notification-preferences	Email opt-in/out
PUT	/users/me/signature	Save or clear signature image
POST	/auth/track-login	Audit login (Clerk optional)
POST	/auth/track-logout	Audit logout
POST	/account/delete-request	Submit GDPR deletion request
GET	/account/delete-request	Deletion request status

Freelancers, portfolio & watchlist

VERB	PATH	PURPOSE
GET	/freelancers	Talent Vault list — filters, <code>?q=</code> , <code>?availableFrom=</code> , profession category, teaching subject; completeness ≥ 60
GET	/freelancers/saved	Personal watchlist — must precede <code>/:id</code>
GET	/freelancers/:id	Freelancer detail including <code>nextAvailableDate</code>
GET	/freelancers/:id/saved	Watchlist membership check
POST	/freelancers/:id/save	Toggle watchlist — 402 at plan limit, 403 for team members
PATCH	/freelancers/:id/watchlist	Update private watchlist notes
GET	/freelancers/me	Own freelancer profile

VERB	PATH	PURPOSE
PATCH	/freelancers/me	Update profile; recalculates completeness atomically; triggers TalentSearch
POST	/freelancers	Create profile; triggers TalentSearch
POST	/freelancers/parse-resume	AI résumé extraction (unmetered)
GET	/public/freelancers/:id	Public profile with portfolio and reviews
GET	/freelancers/:id/portfolio	Public portfolio
GET	/portfolio/me	Own portfolio
POST	/portfolio	Create portfolio item
PATCH	/portfolio/:id	Update portfolio item
DEL	/portfolio/:id	Delete portfolio item
GET	/availability/me	Own blocks — must precede <code>/:freelancerId</code>
POST	/availability/me	Create manual block
DEL	/availability/me/:id	Delete manual block — 409 if booking-derived
GET	/availability/:freelancerId	Public availability PUBLIC

Employers, jobs & interests

VERB	PATH	PURPOSE
GET	/employers/me	Own employer profile
PUT	/employers/me	Update employer profile
GET	/employers/me/candidate-notes/:freelancerId	Candidate hiring file
GET	/job-requirements	Paginated job list
POST	/job-requirements	Create job — FOR UPDATE plan gate; triggers Cruise Mode
GET	/job-requirements/:id	Job detail
PATCH	/job-requirements/:id	Update job (owner only)
DEL	/job-requirements/:id	Delete job (owner only)
POST	/job-requirements/:id/interest	Express interest — plan-limited
GET	/job-requirements/:id/interests	Interest list (employer)
GET	/job-requirements/:id/my-interest	Own interest state

Bookings, milestones & debrief

VERB	PATH	PURPOSE
GET	/bookings	Paginated booking list
POST	/bookings	Create booking — FOR UPDATE plan gate; freezes currency
GET	/bookings/:id	Detail with review, message, hasDebrief

VERB	PATH	PURPOSE
PATCH	/bookings/:id	Update status, milestones, negotiation; triggers debrief on completion
POST	/bookings/:id/negotiate	Propose or counter a rate
GET	/bookings/:id/debrief	Role-filtered debrief slice — 404 until ready
POST	/bookings/:id/debrief	Regenerate — 202, 24 h cooldown
GET	/bookings/:id/milestones	Milestone list
POST	/bookings/:id/milestones	Create milestone
PATCH	/milestones/:id	Update milestone

Agreements

VERB	PATH	PURPOSE
GET	/agreements	Paginated agreement list
POST	/agreements	AI-generate agreement from booking
POST	/agreements/upload-url	Presigned upload URL (PDF/DOCX/TXT)
POST	/agreements/upload-confirm	Confirm, extract text, AI employer summary
PATCH	/agreements/:id/amendments	Update amendment points (uploads only)
POST	/agreements/:id/enrich	AI merge amendments + booking particulars
POST	/agreements/:id/finalize	Health review; unlocks employer signing
GET	/agreements/:id	Detail; <code>employerSummary</code> stripped for non-employers
POST	/agreements/:id/redline	AI edit suggestions
PATCH	/agreements/:id/accept-redline	Accept suggestion; invalidates dependent caches atomically
POST	/agreements/:id/health-score	Five-dimension 0–100 score
POST	/agreements/:id/summarise	Plain-English summary — 403 for employers
POST	/agreements/:id/sign	Sign; role derived server-side
GET	/agreements/:id/download	PDF — 403 <code>NOT_FULLY_SIGNED</code> otherwise

Meetings, messaging & notifications

VERB	PATH	PURPOSE
GET	/meetings	Paginated meeting list
POST	/meetings	Request meeting
GET	/meetings/:id	Detail with brief and disposition when present
PATCH	/meetings/:id	Update status; triggers brief on confirm — does not accept feedback
POST	/meetings/:id/brief	Regenerate brief — 202 Accepted
POST	/meetings/:id/feedback	Disposition + internal notes — 409 if already submitted
POST	/conversations/direct	Find or create a human thread

VERB	PATH	PURPOSE
GET	/conversations/direct	Paginated inbox — <code>?q=</code> , <code>?unread=</code>
GET	/conversations/:id/messages	History; marks read on fetch
POST	/conversations/:id/messages	Send — 429 above 30/hr, 422 on an <code>ai_match</code> thread
PATCH	/conversations/:id/read	Mark thread read
GET	/messages/unread-count	Unread conversation count
GET	/notifications	Paginated notifications
GET	/notifications/unread-count	Unread count — polled every 30 s
PATCH	/notifications/read-all	Mark all read — must precede <code>/:id/read</code>
PATCH	/notifications/:id/read	Mark one read

AI assistance, documents, storage & analytics

VERB	PATH	PURPOSE
GET	/openai/conversations	AI chat list
POST	/openai/conversations	Create AI conversation
GET	/openai/conversations/:id	AI conversation detail
DEL	/openai/conversations/:id	Delete AI conversation
POST	/openai/conversations/:id/messages	Send AI chat message
POST	/ai/match-explanation	Explain a recommendation
POST	/ai/job-description	Generate / improve / check job copy
POST	/ai/proposal	Freelancer proposal drafting
POST	/ai/rate-suggestion	Rate recommendation
POST	/documents/upload-url	Presigned document upload URL
POST	/documents/confirm	Confirm upload; verifies object exists; triggers AI review
GET	/documents/me	Own document statuses with expiry countdown
PATCH	/documents/:documentType/expiry	Set or clear expiry without re-upload
POST	/employer-documents/upload-url	Employer document upload URL
POST	/employer-documents/confirm	Confirm and upsert; triggers AI review
GET	/employer-documents/me	Own statuses — explicit allowlist, never <code>aiNotes</code>
GET	/employer-documents/me/:documentType/view-url	15-minute preview URL
POST	/storage/uploads/request-url	Generic presigned upload URL
GET	/storage/objects/*path	Private object — deny-by-default prefix allowlist
GET	/storage/public-objects/*filePath	Public object <code>PUBLIC</code>
PUT	/storage/local-upload	Local dev signed upload sink

VERB	PATH	PURPOSE
GET	/storage/local-read	Local dev signed read source
GET	/dashboard/stats	Role-specific dashboard metrics
GET	/dashboard/earnings-intelligence	Freelancer earnings analytics
GET	/dashboard/spend-analytics	Employer spend analytics
GET	/dashboard/hiring-analytics	Hiring funnel — <code>?window=30d 90d 12m</code>
GET	/dashboard/activity	Recent activity feed
GET	/analytics/me	Personal analytics rollup
GET	/token-usage/me	Monthly token summary
GET	/token-usage/conversation/:id	Per-conversation breakdown (Growth+)
GET	/subscriptions/me	Plan and usage
POST	/subscriptions/upgrade	Simulated upgrade, Stripe-shaped
GET	/reviews/freelancer/:freelancerId	Public reviews <code>PUBLIC</code>
POST	/reviews	Submit review (completed bookings only)
POST	/reviews/:id/reply	Freelancer reply

Autonomous agents & teams

VERB	PATH	PURPOSE
GET	/talent-search	Config or null <code>EMPLOYER</code>
POST	/talent-search	Create or update config
POST	/talent-search/parse-rules	AI free-form rule parsing
PATCH	/talent-search/activate	Go live; backfills existing profiles
PATCH	/talent-search/dry-run	Evaluate without sending
PATCH	/talent-search/deactivate	Deactivate
GET	/talent-search/activity	Decision ledger — <code>?q=</code> , <code>?decision=</code>
POST	/talent-search/activity/:id/follow-up	Mark follow-up sent
GET	/talent-search/stats	Daily stats and hours consumed
GET	/cruise-mode	Config or null <code>FREELANCER</code>
POST	/cruise-mode	Create or update config
POST	/cruise-mode/parse-rules	AI free-form rule parsing
PATCH	/cruise-mode/activate	Go live — 402 <code>PLAN_LIMIT</code> below Pro
PATCH	/cruise-mode/dry-run	Evaluate without sending
PATCH	/cruise-mode/deactivate	Deactivate and clear

VERB	PATH	PURPOSE
GET	/cruise-mode/activity	Decision ledger
POST	/cruise-mode/activity/:id/follow-up	Mark follow-up sent
GET	/cruise-mode/stats	Daily and monthly stats
POST	/team	Create team ENTERPRISE
GET	/team	Team and member list
PUT	/team	Update team (admin)
POST	/team/invite	Invite by email — single-use 7-day token
GET	/team/accept-invite	Accept invite token
POST	/team/members/:memberId/resend-invite	Resend invite (admin)
DEL	/team/members/:memberId	Remove member — owner cannot be removed
GET	/team/shortlist	Shared shortlist
POST	/team/shortlist	Add to shortlist
DEL	/team/shortlist/:freelancerId	Remove from shortlist
GET	/team/analytics	Team spend and hiring analytics (admin)

Administration **ADMIN** & machine **MACHINE**

VERB	PATH	PURPOSE
POST	/admin/login	Issue tl_admin cookie — rate-limited 10 / 5 min / IP
POST	/admin/logout	Clear admin session
GET	/admin/me	Admin session probe
GET	/admin/csrf-token	CSRF token — registered before the gate
GET	/admin/stats	Platform statistics
GET	/admin/users	User list
GET	/admin/audit	Audit log
GET	/admin/bookings	All bookings
GET	/admin/jobs	All jobs
GET	/admin/subscriptions	Subscription data
GET	/admin/token-usage	Token usage by employer
GET	/admin/documents	Freelancer review queue with signedFileUrl
GET	/admin/documents/:id/signed-url	Signed preview URL
GET	/admin/documents/:id/view-url	Fresh signed URL for fallback opens
PATCH	/admin/documents/:id	Manual verdict; recalculates level; may trigger TalentSearch
GET	/admin/employer-documents	Employer queue — ?status= pending / verified / rejected

VERB	PATH	PURPOSE
GET	/admin/employer-documents/:id/view-url	Signed preview URL
POST	/admin/employer-documents/:id/review	Re-run AI review
POST	/admin/employer-documents/:id/verify	Approve; recalculates level; notifies
POST	/admin/employer-documents/:id/reject	Reject — notes required , else 400
POST	/admin/migrate	Run migration
POST	/admin/seed-demo	Seed demo freelancers
POST	/admin/seed-jobs	Seed demo jobs
POST	/admin/reset-freelancer-availability	Reset availability state
POST	/admin/clear-transactions	Clear transactional data
POST	/admin/reset-test-data	Reset test fixtures
POST	/admin/wipe-all-data	Truncate all tables and restart identities
POST	/cron/credential-expiry	Daily expiry scan — <code>x-cron-secret</code> ; deliberately outside <code>/api/admin</code>

APPX
B

Environment Configuration Matrix

Every variable the runtime reads, its necessity, and the behaviour observed when it is absent.

VARIABLE	NECESSITY	PURPOSE & ABSENT BEHAVIOUR
DATABASE_URL	Required	PostgreSQL connection. Drizzle config throws on startup without it.
PORT	Required	API listen port. <code>index.ts</code> throws — no in-code default.
CLERK_SECRET_KEY	Required	Clerk backend API for auth verification and account deletion.
CLERK_PUBLISHABLE_KEY	Required	Server-side Clerk publishable key, resolved per request host.
VITE_CLERK_PUBLISHABLE_KEY	Required	Client Clerk key, injected at build time.
CSRF_SECRET	Required	Admin double-submit CSRF secret (<code>openssl rand -hex 32</code>). Admin mutations fail without it.
SESSION_SECRET	Required (prod)	Signs the <code>tl_admin</code> cookie. Falls back to an ephemeral per-process secret, invalidating admin sessions on restart.
OPENAI_API_KEY_TALENTLOCK	Required (AI)	Sole OpenAI credential for all 20 inference surfaces. Absent, AI paths fail into their documented fallbacks.
ADMIN_USERNAME	Optional	Admin login username; defaults to <code>admin</code> .
ADMIN_PASSWORD	Conditional	Admin password. Unset disables admin login entirely — fails closed.
ALLOWED_ORIGINS	Required (prod)	CORS allowlist. Development permits all origins when unset.
TRUST_PROXY	Optional	Express trust-proxy hop count; needed for correct client IPs behind a load balancer.
CRON_SECRET	Required (cron)	Shared secret for <code>x-cron-secret</code> . Route returns <code>500</code> when unset — never an unauthenticated success.
PREMIUM_FEATURES_FREE	Flag	Currently on. Bypasses every plan and token gate server-side.
VITE_PREMIUM_FEATURES_FREE	Flag	Client twin of the above; opens UI-level plan gates.
AZURE_STORAGE_CONNECTION_STRING	Conditional	Azure Blob backend for documents and agreements.
DEFAULT_OBJECT_STORAGE_BUCKET_ID	Conditional	GCS bucket identifier.
PRIVATE_OBJECT_DIR	Conditional	Private object prefix resolved by the storage facade.
PUBLIC_OBJECT_SEARCH_PATHS	Conditional	Search paths for publicly served objects.
LOCAL_OBJECT_STORAGE_DIR	Optional	Enables the local disk backend with self-signed HMAC URLs — no cloud credentials required in development.
RESEND_API_KEY	Optional	Transactional email. Silent no-op when unset; in-app notifications continue.
EMAIL_FROM	Optional	Sender address; defaults to <code>noreply@talentlock.io</code> .
APP_URL	Optional	Base URL for email deep links; defaults to <code>http://localhost:25807</code> .
API_PROXY_TARGET	Optional	Vite dev proxy target for <code>/api</code> .
API_PUBLIC_URL	Optional	Absolute API base used when constructing storage links.

VARIABLE	NECESSITY	PURPOSE & ABSENT BEHAVIOUR
BASE_PATH	Optional	Client base path; defaults to <code>/</code> .
VITE_CLERK_PROXY_URL	Conditional (prod)	Clerk Frontend API proxy URL paired with <code>/api/__clerk</code> .
ENABLE_DEMO_LOGIN	Flag	Enables demo session minting. Must be <code>false</code> in production.
DEMO_FREELANCER_CLERK_ID DEMO_EMPLOYER_CLERK_ID	Optional	Fixed Clerk ids for demo accounts.
DATABASE_URL_TEST	CI	Integration database; mapped to <code>DATABASE_URL</code> in the CI job.
LOG_LEVEL	Optional	Pino verbosity.
E2E_BASE_URL E2E_SKIP_WEBSERVER	Optional	Playwright targeting; suppresses auto-start of the dev server.

ARCHITECTURAL CONSTRAINT — FAIL-CLOSED SECRETS

Three secrets deliberately fail closed rather than degrading to an open state: `ADMIN_PASSWORD` (admin login disabled), `CRON_SECRET` (cron route returns `500`), and `CSRF_SECRET` (admin mutations rejected). This is the correct posture — an unconfigured deployment is inert, not publicly writable.

APPX C

Verification & Quality Posture

Automated test composition, continuous integration topology, and an objective assessment of architectural strengths against open risks.

Test composition

LAYER	SUITES	GATING	COVERAGE FOCUS
Server unit	23	Always run	Pure logic — plan gating, pre-filters, expiry-stage math, completeness scoring, currency, pagination, prompt builders, access-control predicates, cron auth
Server integration	47	Skipped without live env	Supertest against the real Express app — IDOR matrix, admin CSRF, sanitisation, quota 402s, agreements, meetings, messaging, agents, cron, watchlist, GDPR deletion
Client unit / component	8	Always run	Testing Library — completeness banner, pagination controls, verified badge, plan-error routing, rate formatting, contract-health display
End-to-end	1	CI job	Playwright smoke journey on Chromium

TECHNICAL NOTE — INTEGRATION GATING

Every integration suite is wrapped in `describe.skipIf(!integrationEnvReady())`, which requires a `real DATABASE_URL` (a placeholder value is explicitly detected and rejected) plus Clerk credentials. Admin sub-suites additionally require `ADMIN_PASSWORD`. The consequence: a local `pnpm test` reports **99 passing and 185 skipped**. Full-surface verification only occurs in the nightly and `main`-push CI job where secrets are present — worth knowing before treating a green local run as comprehensive.

Continuous integration topology

01 Install --frozen-lockfile	02 Typecheck All 11 packages	03 Unit Server + client	04 E2E Playwright Chromium	05 Schema push Test database	06 Integration Nightly + main only
---	---	--------------------------------------	--	---	---

Two workflows are defined. `test.yml` runs on every pull request and push to `main`, plus nightly at 03:00 UTC, emitting JUnit artefacts. Its integration job is restricted to nightly and `main` pushes because it requires database and Clerk secrets. `credential-expiry-cron.yml` runs daily at 07:00 UTC and is the production trigger for the expiry scan — it is operational infrastructure, not a test.

Assessment

DIMENSION	STANDING	EVIDENCE
Contract integrity	Strong	Single OpenAPI source generating both client hooks and server validators; drift is structurally prevented rather than policed.
Transactional correctness	Strong	Row-level locks on every quota path; derived columns written in the same statement as their source; verification levels recalculated inside the triggering transaction.
Failure isolation	Strong	Uniform fire-and-forget egress; AI, email, and storage failures cannot fail a domain write. Every AI path has a defined degradation.
Authorisation rigour	Strong	Per-resource participation checks with a consistent 401/403/404 convention, validated at 32/32; deny-by-default private object serving.

DIMENSION	STANDING	EVIDENCE
Documentation discipline	Strong	37 specification directories and a 750-line architecture reference carrying explicit anti-patterns and route-ordering constraints.
Observability	Partial	Structured logs, decision ledgers, token attribution, and audit trail are present; APM, tracing, error aggregation, and uptime metrics are not.
Schema governance	Partial	Declarative <code>push</code> with no migration history. Mitigated by the <code>verify-schema</code> parity guard, but reproducing a historical schema state is not possible from the repository alone.
Request rate control	Gap	No global limiter. Rate limits exist only on admin login and direct messages; AI endpoints are unbounded per unit time, and token quotas are currently bypassed.
Inference accounting	Gap	<code>routes/resumeParser.ts</code> calls <code>gpt-4o</code> without <code>LogTokenUsage</code> — the sole unmetered surface of twenty-one.
Revenue readiness	Gap	Billing is simulated; no payment processor is wired. <code>PREMIUM_FEATURES_FREE</code> holds every entitlement gate open.

Recommended sequence before paid launch

1. **Close the metering gap** — add `LogTokenUsage` to the résumé parser with a dedicated feature label, restoring complete cost attribution.
2. **Introduce a global rate limiter** in front of `/api/ai/*`, `/api/openai/*`, and the agreement AI routes, so cost is bounded by rate as well as by monthly quota.
3. **Flip the entitlement flags** — set `PREMIUM_FEATURES_FREE` and `VITE_PREMIUM_FEATURES_FREE` to `false`, restore list prices, and verify the 402 paths end to end against the existing integration suite.
4. **Wire the payment processor** into the already Stripe-shaped upgrade surface and the reserved `subscriptions` columns.
5. **Instrument the runtime** — error aggregation and uptime monitoring at minimum, since diagnosis currently depends on manual log inspection.
6. **Remove or mount** `routes/verifyDocuments.ts` to eliminate ambiguity about whether an unreferenced route surface is intended to exist.

CLOSING ASSESSMENT

TalentLock presents as an unusually disciplined codebase for its feature density: 48 domains, 179 endpoints, and 20 metered inference surfaces held together by a generated API contract, a uniform request pipeline, and consistent transactional and deferred-effect patterns. The architecture is coherent and the invariants are documented and enforced rather than merely asserted. The open items are **operational rather than structural** — rate limiting, runtime instrumentation, one metering omission, and commercial activation. None require architectural change; all are additive to the existing seams.